

Words Towards Explainability: Caption Label-Free Learning via Dual Loop Agentic Time Series Captioning

Difei Hou¹ Jiaqi Yue¹ Chunhui Zhao¹

Abstract

Explainability is essential for applying time series analysis in high-stakes domains. While Time Series Captioning (TSC) offers a pathway to enhance temporal explainability, achieving reliable caption generation usually necessitates high-quality textual annotations. However, as interpreting abstract temporal dynamics requires specialized domain knowledge, acquiring such caption annotations is challenging, thereby impeding the advancement of TSC. To address this challenge, we introduce a novel Caption Label-Free Learning (CLFL) paradigm. Departing from the supervised learning tradition of imitating human annotations, CLFL formulates captioning as an agentic exploration task optimized by feedback from a proxy reward. Specifically, we propose a Dual Loop Agentic Captioning (DLAC) framework to achieve such an exploration-feedback mechanism. In the inner loop, a Time Series Captioning Agent (TSCAgent) reflectively explores potential semantic captions. In turn, the outer loop evaluates these captions via downstream reasoning to derive proxy reward, which feeds back to optimize the TSCAgent. Empirical results validate the effectiveness of the CLFL, proving that the exploration-feedback mechanism is sufficient for learning complex temporal semantics and autonomously generating captions, without any caption label supervision. Furthermore, we release TFTSC, an industrial expert-level time series caption dataset, which is available at: <https://github.com/chunhuiz/TFTSC/tree/master>.

¹State Key Laboratory of Industrial Control Technology, College of Control Science and Engineering, Zhejiang University, China. Correspondence to: Chunhui Zhao <chzhao@zju.edu.cn>.

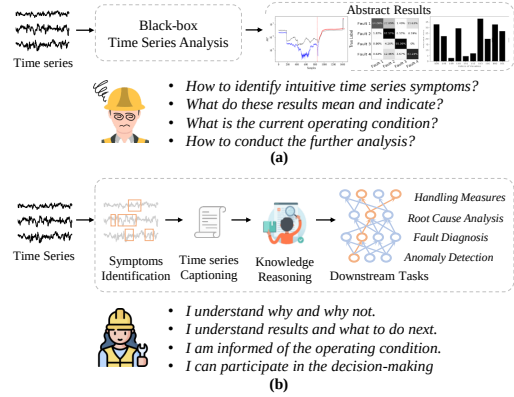


Figure 1. Considering a novel explainable analysis process enabled by time series captions, extending from time series data understanding to decision-making. (a) Existing methods struggle to provide additional explanations beyond numerical values. (b) Captions possess readability and facilitate human-comprehensible downstream reasoning.

1. Introduction

Time series analysis is ubiquitous in domains such as industrial manufacturing (Zhao et al., 2025a; Liu et al., 2024), finance (Belkhouja et al., 2022; Gao et al., 2024), and healthcare (Moor et al., 2023; Raghu et al., 2023). While deep learning has substantially advanced modeling capabilities, mainstream approaches predominantly map physical signals into abstract latent representations, such as high-dimensional feature vectors (Troncoso-García et al., 2025; Wang et al., 2024b). Despite their superior predictive performance, these opaque representations lack semantic interpretability and logical transparency (Wang et al., 2022). Consequently, deciders in critical domains such as industry, finance, and healthcare, often remain skeptical of analysis models, hindering their deployment in high-stakes decision-making scenarios. Therefore, it is highly necessary to enhance the explainability of the temporal analysis model.

Existing research on eXplainable Artificial Intelligence (XAI) for time series analysis faces distinct challenges compared to other modalities. Mainstream approaches primarily rely on feature attribution methods such as LRP (Vielhaben et al., 2024), LIME (Sivill & Flach, 2022), and SHAP

(Zhang et al., 2023). Attribution is a post hoc analysis that decomposes input features into intelligible components and evaluates the contribution of each component to the output, to interpret the decision process of the model (Arrieta et al., 2020; Bello et al., 2025). In the field of such as Computer Vision (CV) (Li et al., 2024; Adebayo et al., 2018) or Natural Language Processing (NLP) (Yuan et al., 2022), semantic units intelligible to humans can be readily extracted from data, such as visual objects or linguistic tokens. However, the extraction of such semantic units from abstract time series data remains an open challenge (Sivill & Flach, 2022; Ismail et al., 2020). The meaning of attribution results is often ambiguous and requires specialized domain knowledge. Consequently, it is difficult for users to understand.

In contrast to attribution methods, translating salient time series patterns (e.g., abrupt changes or steady states) into linguistic captions is a promising approach for enhancing explainability. This is because the readability of natural language significantly reduces cognitive load and establishes user trust more reliably than abstract numerical representations (Sweller, 1988; Miller, 2019). In terms of understanding transparency, high-quality Time Series Captioning (TSC) promotes explainability throughout the entire analysis process. This is illustrated by an industrial example in Figure 1. While early methods relied on rigid rule-based templates (Yu et al., 2003), recent research in TSC focuses on data-driven paradigms to generate more flexible descriptions (Jhamtani & Berg-Kirkpatrick, 2021). However, these supervised approaches rely heavily on extensive expert annotations (Liu et al., 2024). Due to the complex dynamics and multivariate correlations inherent in time series, obtaining meaningful descriptions requires extensive domain expertise (Feng & Zhao, 2020; Zheng et al., 2023), which creates a bottleneck in label availability.

Given the scarcity of time series caption annotations, it is imperative to investigate new learning frameworks distinct from conventional supervised learning. We observe that time series captions can serve as the semantic foundation for downstream reasoning, which enables tasks such as time series classification. Intuitively, if time series captions are uninformative or contain excessive noise, downstream reasoning cannot derive correct conclusions. Motivated by this insight, we introduce an exploration-feedback mechanism, where the an agentic exploration for caption generation is optimized by feedback of proxy reward from the downstream reasoning. Moreover, to enhance flexibility and controllability in caption generation, we propose disentangling semantic content (“what to say”) from linguistic realization (“how to say it”). Specifically, we predefine multiple deterministic descriptive approaches for linguistic realization. Then, a central agent is employed to intelligently and reflectively select suitable descriptive perspectives for different time series segments.

Motivated by the aforementioned analysis, we introduce Caption Label-Free Learning (CLFL), a novel paradigm that shifts the time series captioning optimization objective from imitating static human labels to an autonomous exploration-feedback process. To materialize this, we develop a Dual Loop Agentic Captioning (DLAC) framework. In the inner loop, a Time Series Captioning Agent (TSCAgent), based on a Large Language Model (LLM), reflectively explores critical temporal patterns and generates captions using descriptive tools. In turn, the outer loop derive proxy reward by evaluating these captions via downstream reasoning, which is semantic coverage-guaranteed. This proxy reward provides feedback to optimize TSCAgent by reinforcement learning (RL) to autonomously generate specialized captions, thereby enabling transparent and explainable analysis. Furthermore, we release Three-phase Flow Time Series Captions (TFTSC), an industrial expert-level TSC dataset. This benchmark encompasses more than 60k time series samples across 4 system operating states and 11 sensor channels, with captions covering 14 major pattern categories. In summary, our contributions are follows:

1. We introduce a Caption Label-Free Learning (CLFL) paradigm to eliminate reliance on time series caption annotations. Different from conventional supervised learning, CLFL reframes captioning as an agentic exploration task optimized by feedback from proxy reward, enabling the model to autonomously acquire temporal semantics.
2. We propose Dual Loop Agentic Captioning (DLAC) to establish an exploration-feedback mechanism. In the inner loop, a TSCAgent reflectively explores potential critical temporal dynamics and forms captions. In turn, the outer loop evaluates generated captions via downstream reasoning, to feed back proxy rewards for TSCAgent RL optimization.
3. We release an industrial expert-level Time Series Captions dataset, TFTSC, from the three-phase flow processes. This dataset comprises over 60,000 time series samples paired with captions of multiple descriptive perspectives, which serves as a novel benchmark for time series caption generation.

2. Related Work

2.1. Time Series Captioning

Translating time series data into natural language descriptions remains a challenging endeavor, primarily due to the paucity of textual annotations and inherent technical complexities. Early approaches relied on domain-specific rules to identify time series patterns and populate them into pre-defined description templates (Yu et al., 2003). However,

these rules lack the flexibility to generalize across different specialized scenarios (Zhao et al., 2025b; Yang et al., 2025). While subsequent data-driven studies have attempted to map time series directly to linguistic patterns such as trends or fluctuations (Sowdaboina et al., 2014; Jhamtani & Berg-Kirkpatrick, 2021), these fully supervised paradigms necessitate extensive high-quality text annotations, which are unfortunately expensive and rare (Trabelsi et al., 2025). To mitigate annotation paucity, Xie et al. (Xie et al., 2025) synthesized pairs of time series and text, which is simulated through the mathematical superposition of modules such as linear growth and sine waves, and annotated by modules names. Nevertheless, generating high fidelity samples with real-world characteristics for specialized domains, which often involve noise and irregular sampling (Ansari et al., 2023), remains a non-trivial task (Ren et al., 2025).

2.2. Time Series Analysis based on LLMs

The Large Language Models (LLMs) possess the capability to follow instructions and can output their understanding of time series data in textual format (Petruzzelli et al., 2024; Slack et al., 2023). Some studies have attempted to input time series numbers directly into LLMs as textual prompts and utilize prompt words to perform time series analysis (Fons et al., 2024; Li & Zhao, 2025). However, the practicality of time series analysis based on LLMs remains questionable due to hallucination (Huang et al., 2025; Jesson et al., 2024; Wang et al., 2024a). Fundamentally, the contextual dependencies captured by LLMs (Zhang et al., 2024), such as word co-occurrence, syntax, and semantic associations, diverge significantly from the temporal dynamics like trends, seasonality, and correlations, which require precise numerical calculations and are a known weakness (Jiang et al., 2025; Li et al., 2025) of current LLMs. Existing research demonstrates that converting time series into line charts for input into multimodal LLMs does not yield significant improvements (Zhou et al., 2025). Furthermore, time series data is abstract, and the underlying generation mechanism is highly specialized, which general LLMs struggle to comprehend (Yang et al., 2023; Tan et al., 2024). Consequently, given the paucity of time series caption annotations, fine-tuning LLMs is unfeasible. As a result, the quality of LLMs generated time series descriptions remains uncontrollable.

3. Methodology

In this section, we successively present the problem formulation of the time series captioning, and present the overall framework of the proposed DLAC. Subsequently, we detail the exploration in the inner loop and the feedback in the outer loop, respectively. Finally, we analyze the applicability of the proposed method across diverse domains.

3.1. Problem Formulation

Let $\mathbf{X} \in \mathbb{R}^{m \times L}$ denote a multivariate time series segment consisting of m variables with length L . To ensure the segment encapsulates sufficient semantic information for interpreting system dynamics, we define the minimum segment length as a guarantee of semantic validity, denoted as L_{min} . Segments with length $L < L_{min}$ may suffer from semantic ambiguity due to incomplete observation of temporal patterns. While L_{min} can be empirically determined by domain priors, we propose an adaptive estimation approach applicable to diverse datasets (details in Appendix A.2).

The domain-specific knowledge for captioning and downstream reasoning is formally denoted as K , which comprises system knowledge K_{sys} and task knowledge K_{task} . Specifically, K_{sys} encompasses system background, dynamic mechanisms, variable semantics, and intervariable correlations. In contrast, K_{task} pertains to operational contexts and unstructured expert logs, which typically reside in external documentation (See Appendix C.3). The captioning problem is defined as generating a textual description o given $\mathbf{X}$ and K . We denote the proposed TSCAgent as $\mathcal{F}$, and operations involving text comprehension, summarization, and knowledge reasoning are executed by a separate LLM, denoted as $\mathcal{G}$. The generation process can be abstracted as $o = \mathcal{F}(\mathbf{X}, K; \mathcal{G})$.

3.2. Framework Overview

The proposed DLAC implements an exploration-feedback process through the interaction between an inner loop and an outer loop, as illustrated in Figure 2. **1)** In the inner loop, a TSCAgent is equipped with a specialized captioning toolset that includes multiple descriptive perspectives. The exploration is realized through a reflective agent workflow. Within this workflow, the TSCAgent autonomously selects appropriate perspectives for distinct time series segments. Furthermore, the agent performs self-reflection according to retrieved information. **2)** The generated captions support downstream reasoning in the outer loop to produce a proxy reward for the TSCAgent. This feedback relies on the effectiveness of the downstream reasoning. Given that captions function as the semantic foundation, they should possess complete semantic information. Therefore, prior to the reasoning, we propose a semantic coverage assurance scheme based on a theoretical analysis of temporal semantic evolution.

3.3. Exploration in Inner Loop

In this section, we sequentially introduce the proposed specialized captioning toolset and the reflective agent workflow of TSCAgent.

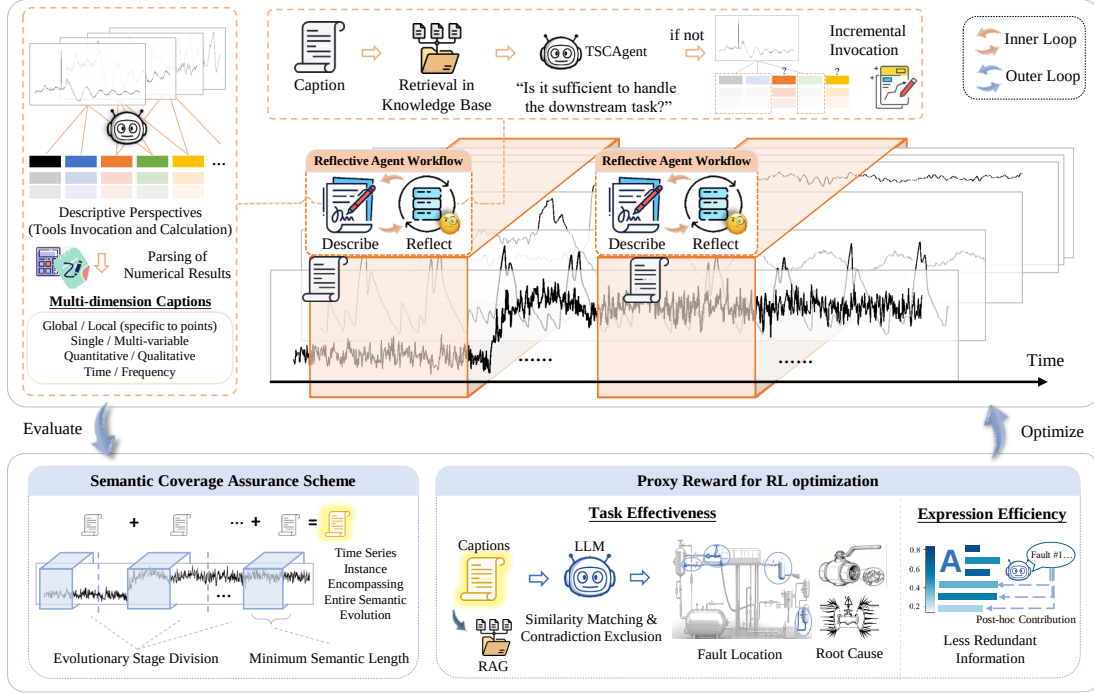


Figure 2. Our proposed DLAC integrates the inner loop (Orange Arrow) and the outer loop (Blue Arrow). **Top left:** In the describing phase, the TSCAgent selects appropriate perspectives for the given time series segment and invokes tools to generate captions. **Top right:** The module reflects on the captions produced during the describing phase. **Down left:** This step ensures semantic coverage before the downstream reasoning process. **Down right:** The proxy reward is calculated based on performance in downstream tasks to guide the optimization of the TSCAgent.

3.3.1. SPECIALIZED CAPTIONING TOOLSET

We develop a toolset $\mathcal{T}$ that instantiates captioning perspectives into actionable utilities, enabling the TSCAgent to invoke distinct tools tailored to specific time series characteristics. These provide not merely a holistic description but also precise timestamps for specific events, such as spikes and steps, through predefined rules.

The $\mathcal{T}$ comprises 14 major categories and 42 distinct types of time series descriptive tools (See Table 5 in Appendix A.1 for details.). Facilitated by these predefined tools, captions incorporate multiple descriptive dimensions including *Global versus Local*, *Quantitative versus Qualitative*, *Single versus Multivariable*, and *Time versus Frequency* domains. Furthermore, we formulate numerical parsing instructions P for each computational result. Under the guidance of P , $\mathcal{G}$ generates further qualitative analysis of the computational results, thereby reducing the understanding difficulty of subsequent reasoning.

3.3.2. REFLECTIVE AGENT WORKFLOW

The proposed workflow involves the TSCAgent that alternates between the *Describing* and *Reflecting* phases.

Describing: For a given X , if domain-specific prior knowl-

edge permits the selection of key variables, only those variables are described (See Appendix A.4); otherwise, the description encompasses all m channels. The input to the *Describing* phase comprises the time series data of each channel alongside the corresponding K_{sys} and K_T . Consequently, $\mathcal{F}$ identifies noteworthy *Semantics* within the time series and generates M tool calling commands $A = \{a_1, a_2, \dots, a_M\}$, where each command explicitly specifies the *tool name* and the *corresponding variable*. After A are executed, we parse the returned contents $C = \{c_1, c_2, \dots, c_M\}$ with P to obtain the full caption o .

Reflecting: The obtained caption o is utilized to retrieve relevant information to incorporate task knowledge into the prompts, denoted as $R(o, K_{\text{task}})$. The input to the *Reflecting* phase comprises the o of all channels and the retrieved information $R(o, K_{\text{task}})$. Guided by prompts, $\mathcal{F}$ conducts a critical reflection. Should $\mathcal{F}$ determine that the accumulated information is insufficient for the downstream task, it identifies the specific deficiency $p_{\text{shortcoming}}$ and reverts to the *Describing* phase. The *Describing* phase subsequently addresses $p_{\text{shortcoming}}$ by incrementally selecting alternatives while excluding prior invocations. This iteration persists until a predefined iteration limit is reached or $\mathcal{F}$ attains sufficient confidence and then generates the final caption.

3.4. Feedback in Outer Loop

In the outer loop of DLAC, we perform downstream reasoning with a semantic coverage assurance scheme. The reasoning results generate proxy reward for TSCAgent optimization.

3.4.1. SEMANTIC COVERAGE ASSURANCE SCHEME

We employ a theoretical semantic coverage assurance scheme before the reasoning phase. The captioning process of TSCAgent applies to any input X with a length exceeding L_{min} . However, intrinsic information in X may only include local dynamics. This is often insufficient for downstream reasoning and fails to match the temporal global semantic evolution, leading to decision ambiguity or erroneous judgments. To address this, we leverage an optimal partition strategy to divide evolutionary stages. Subsequently, we sample X within each partitioned stage and aggregate the captions of these windows, ultimately deriving a semantic coverage with a lower bound.

Formally, let $X^{\text{full}} \in \mathbb{R}^{m \times T}$ denote the time series instance encompassing the entire semantic evolution. The *semantic evolution velocity* and *semantic evolutionary arc length* are defined to partition X^{full} into distinct evolutionary stages. We prove that, provided information in X sampled from each partitioned evolutionary stage is incorporated, the total semantic omission is constrained by a finite upper bound, thereby guaranteeing a lower bound for semantic coverage. We define a mapping $\Phi : \mathcal{X} \rightarrow \mathcal{P}$ that projects an arbitrary X_t of length L_{min} onto a point P_t on a statistical manifold $\mathcal{M}$. Here, Φ can be instantiated as any pretrained encoder. Let $\text{dist}_{\mathcal{M}}(\cdot, \cdot)$ denote the Riemannian distance induced by the Fisher-Rao metric (Rao et al., 1945) on $\mathcal{M}$.

Definition 3.1. On the statistical manifold $\mathcal{M}$, we define the instantaneous semantic evolutionary velocity between time t and $t + \delta$ as follows:

$$v(t) = \lim_{\delta \rightarrow 0} \frac{\text{dist}_{\mathcal{M}}(P_t, P_{t+\delta})}{\delta} \quad (1)$$

Based on the semantic evolutionary velocity field $v(t)$, the semantic evolutionary arc length from $t = 0$ to the current time t is denoted as $\Lambda(t)$ and calculated as:

$$\Lambda(t) = \int_0^t v(\tau) d\tau \quad (2)$$

where $\Lambda(T)$ represents the entire semantic evolutionary arc length on the semantic manifold, which is a finite value.

Under the settings of a fixed number of evolution stages n and sampled once in each stage, we define the semantic omission and propose an optimal evolutionary stage partitioning strategy.

Definition 3.2. Let the trajectory of time series data mapped onto the statistical manifold $\mathcal{M}$ be the curve $P(t)$, where $t \in [0, T]$. For the k th stage among n evolution stages (time interval $[t_{k-1}, t_k]$) and a selected point $d_k \in \mathcal{M}$ on the arc, we define the semantic omission E_k for the k th stage and the total semantic omission for the entire process as

$$\mathcal{E} = \sum_{k=1}^n E_k = \sum_{k=1}^n \int_{t_{k-1}}^{t_k} \text{dist}_{\mathcal{M}}^2(P(t), d_k) dt \quad (3)$$

Proof. See Appendix B.1.

Remark 3.3. The optimal evolutionary stage partitioning strategy is equidistant dividing on the semantic manifold, given by

$$l_k = \frac{\Lambda(T)}{n} \quad (4)$$

where l_k represents the arc length for each stage. This optimal partitioning strategy minimizes the semantic omission of the entire process. If the time series segment X corresponding to the centroid position within each evolution stage is sampled, under the local tangent space approximation, the ideal minimum semantic omission $\mathcal{E}$ is obtained as

$$\mathcal{E}_{\text{opt}} = \frac{1}{12} \frac{\Lambda(T)^3}{n^2} \quad (5)$$

where $\mathcal{E}_{\text{opt}}$ converges at a quadratic rate with the increase of the preset number of stages n .

Proof. See Appendix B.2.

Remark 3.4. With arbitrary sampling within stages, there exists a finite upper bound for the semantic omission in the worst case as

$$\mathcal{E}_{\text{worst}} = 4 \cdot \mathcal{E}_{\text{opt}} \quad (6)$$

Therefore, aggregation under the optimal evolutionary stage partitioning strategy contributes to the semantic coverage with a lower bound.

Technically, we perform equidistant partitioning on $\Lambda(T)$ and map the intervals back to the temporal domain to implement stage division. For a given X^{full} , we calculate the semantic evolutionary velocity by computing the distance between vectors of adjacent X . Then, we obtain $\Lambda(t)$ by accumulating these velocities up to time step t . We then divide $\Lambda(T)$ by l_k into equal intervals and map these points back to the nearest temporal indices. Finally, from each evolutionary stage, we sample an X , aggregate their captions, and proceed with the downstream task.

3.4.2. PROXY REWARD FOR RL OPTIMIZATION

The proxy reward $\mathcal{R}$ comprises a *Task Effectiveness Reward* R_{te} and an *Expression Efficiency Reward* R_{ee} to optimize caption generation from two perspectives, which provides comprehensive and more granular optimization signals.

Task Effectiveness Reward: We utilize the fault diagnosis task in industrial processes as an exemplar to introduce the downstream reasoning. In the context of time series classification tasks such as fault diagnosis, text-based reasoning is essentially a matching process between captions and K_{task} of different faults. Technically, we utilize LLMs to transform the above matching process into a RAG-based question answering (Q&A) process as

$$\hat{y} \sim \mathbb{P}_{\mathcal{G}}(\cdot \mid o \oplus R(o, K_{\text{task}})) \quad (7)$$

where $\oplus$ denotes concatenation of the texts, and $\hat{y}$ is the predicted fault category, with which we compare the downstream task labels y . Strategies such as changing different LLMs $\mathcal{G}$ and different sampling temperature settings can be adopted to obtain multiple sampling results for the same o , and we count the proportion of the correct as the *Task Effectiveness Reward* R_{te} .

Expression Efficiency Reward: We expect each agent-invoked tool to contribute meaningfully to the downstream task, ensuring that the generated captions contain no redundant information. Therefore, we employ Integrated Gradients, a prevalent attribution technique based on gradients (Sundararajan et al., 2016), to quantify the contribution score of textual segments returned by various tools to the task result generated by $\mathcal{G}$. We denote the minimum contribution among all textual segments as $Con_{\min}$, which is normalized and constitutes a component of the *Expression Efficiency Reward* R_{ee} . More calculation details are provided in the Appendix A.3.

3.5. Discussion of CLFL Applicability

The proposed CLFL paradigm extends its applicability far beyond the aforementioned industrial fault diagnosis scenarios. Conceptually, its validity relies on utilizing proxy rewards obtained from downstream reasoning to optimize time series caption generation. Consequently, CLFL is structurally applicable whenever a generated caption can lead to a verifiable conclusion. Methodologically, the paradigm is intrinsically agnostic to specific domains and downstream reasoning tasks. (1) *This paradigm inherently supports broad applicability across diverse domains.* In fields such as finance and healthcare, time series captions naturally serve as the foundation for subsequent decision-making. (2) *The formulation of CLFL is unrestricted by the specific type of downstream task.* Whether the objective is temporal classification, forecasting, or causal inference, the optimization of caption generation remains feasible as long as the downstream task can yield a quantifiable proxy reward reflecting the utility of the generated caption, which is the only prerequisite of the CLFL. Further discussion details are provided in the Appendix A.5.

4. Experiment

4.1. Experimental Settings

The proposed CLFL is validated on the three-phase flow industrial process, which serves as a widely adopted benchmark for fault diagnosis in the process industry (Chai & Zhao, 2020). This time series dataset encompasses 4 balanced fault types, which can be further subdivided into 10 fine-grained fault cases (Ruiz-Cárcel et al., 2015) (More details see Appendix C). We uniformly sampled time series windows from the dataset, yielding over 3500 multivariate samples annotated with fault categories. The length of each window exceeds the minimum semantic length $L_{\min}$. We utilize 1500 samples for training and employ GRPO (Shao et al., 2024) method to finetune the base model. We adopt the base models *Qwen2.5-7B-Instruct* and *Qwen2.5-VL-7B-Instruct* (Team, 2025a) for TSCAgent $\mathcal{F}$, and select *Qwen2.5-14B-Instruct* as $\mathcal{G}$. Additional details are provided in the Appendix D.1.

We review strong baselines currently available for time series captioning, which are categorized into three groups: approaches utilizing LLMs such as Gemini 3 Pro (Gemini Team, Google DeepMind, 2025) for direct time series question answering; approaches leveraging Multimodal LLMs capable of visual interaction, such as GPT-4o (Achiam et al., 2023), which accept both time series line charts and numerical values as input; and Caption label supervised captioning methods including Time-MQA (Kong et al., 2025), ChatTS (Xie et al., 2025), and ITFormer (Lei et al., 2025). The baselines list and more details in the comparison experiment are in Appendix D.2.

4.2. Results

Caption Quality Evaluation: We uniformly selected 100 samples and invited domain experts to annotate a gold standard dataset D (See Appendix D.3). We evaluate the generated captions using standard linguistic metrics, including ROUGE-L (Lin, 2004), BERTScore (Zhang* et al., 2020), and FActScore (Min et al., 2023), while quantifying numeric fidelity via Numeric-Score (Zhou et al., 2025). The Numeric-Score is calculated based on the matching between numerical values from the reference and the generated text (See Appendix D.4).

The results presented in Table 1 demonstrate that the captions generated by our method operate without caption annotations, yet achieve superior performance across all metrics (“Ours(Qwen2.5-VL-7B)”). In particular, it yields a 16.6% improvement in FActScore compared to the second-best method, which stems from the reflective extraction of sufficient temporal patterns within the inner loop. Furthermore, TSCAgent maintains numeric fidelity by performing calculations on raw values through tool invocation, notably yielding

Table 1. Textual quality of time series captions generated by our method compared to three categories of strong baselines. These categories include approaches utilizing LLMs, Multimodal LLMs, and existing caption label supervised methods. “as Agent” indicates the model participates in a reflective agent workflow, while not fine-tuned. “RO”, “BE”, “FA”, and “NU” indicate the ROUGE-L metric, BERTScore, FactScore, and Numeric-Score, respectively. A higher value of metric $\uparrow$ indicates better performance. The highest is in **bold**, and the second best is underlined. “Improvement” indicates improvement of our method on different metrics over the second-best method.

Method		RO $\uparrow$	BE $\uparrow$	FA $\uparrow$	NU $\uparrow$
LLMs	Gemini 3 Pro	0.194	0.640	0.561	0.601
	GPT-5.2	0.247	0.625	0.738	0.717
	Deepseek-V3.2	0.299	0.542	0.634	0.664
	Qwen3-235B	0.202	0.599	0.450	0.576
Multimodal LLMs	GPT-4o	0.317	0.653	0.711	0.637
	Qwen3-VL	0.187	0.603	0.682	0.528
	Seed1.6-VL	0.304	0.624	0.592	0.509
	GLM4.5V	0.260	0.631	0.431	0.572
Caption label supervised methods	Time-MQA	0.298	0.614	0.683	0.471
	ChatTS	0.273	0.633	0.739	0.730
	ITFormer-7B	0.301	0.712	0.783	0.673
Ours	GPT-5.2 as Agent	0.318	0.779	0.813	0.796
	GPT-4o as Agent	0.320	0.808	0.820	0.751
	Qwen2.5-7B as Agent	0.161	0.424	0.102	0.626
	Qwen2.5-VL-7B as Agent	0.154	0.459	0.081	0.647
	Ours(Qwen2.5-7B)	0.326	0.811	0.899	0.883
	Ours(Qwen2.5-VL-7B)	0.330	0.826	0.913	0.909
Improvement		4.1%	16.0%	16.6%	24.5%

a 24.5% improvement in numeric accuracy compared to the second-best method. Moreover, we selected the competitive GPT-5.2 and GPT-4o models to test the performance improvement after being equipped with the toolset, allowing it to participate in the reflective agent workflow in DLAC, and it is recorded as “as Agent”. Empirical results shows that predefined descriptive perspectives in the inner loop effectively facilitate alignment with human expert annotations. For instance, the “GPT-5.2 as Agent” configuration achieves a 28.7% improvement in ROUGE-L compared to the GPT 5.2 baseline.

Caption Utility in DownStream Task: Excluding the training set, we utilize the remaining multivariate time series window samples to test caption utility in the downstream task. As in the previous experimental setup, we selected several competitive models to participate in the reflective agent workflow in DLAC to generate captions. The captions are aggregated via a semantic coverage assurance scheme before the downstream reasoning. The proportion of correct diagnoses across multiple samplings is defined as *Diagnosis Accuracy* metric, and the Con_{min} is termed as *Expression Conciseness* metric. This configuration ensures that all methods access the same descriptive perspectives to validate the superiority of the feedback in the outer loop.

Experimental results presented in Table 2 demonstrate that, first, our method generates captions with the highest utility in the downstream task, outperforming the second-best

Table 2. Caption utility in downstream tasks compared to competitive baselines. “as Agent” indicates the model participates in reflective agent workflow, while not fine-tuned. The higher metrics are better, best in **bold**, and second best underlined. “Improvement” indicates improvement of our method on different metrics over the second-best method.

Method	Diagnosis Accuracy	Expression Conciseness
GPT-5.2 as Agent	0.8211	0.0466
GPT-4o as Agent	0.8225	0.0463
Qwen3-VL as Agent	0.5882	0.0203
Seed1.6-VL as Agent	0.7058	0.0427
GLM4.5V as Agent	0.6471	0.0270
Qwen2.5-7B as Agent	0.3647	0.0010
Qwen2.5-VL-7B as Agent	0.3976	0.0022
Ours(Qwen2.5-7B)	0.8744	0.0489
Ours(Qwen2.5-VL-7B)	0.9743	0.0519
Improvement	18.5%	11.4%

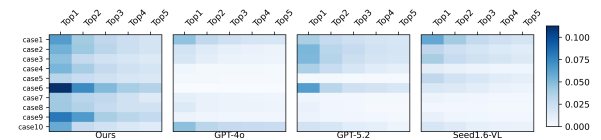


Figure 3. Visualization of contribution scores for the top-5 segments across 10 cases. The heatmap color depth indicates the score value, where darker regions correspond to higher contribution.

method by 18.5%. Second, while the original base model “Qwen2.5-VL-7B” exhibits limited proficiency in time series captioning, RL optimization via the proxy reward significantly enhances its capability, yielding a significant performance gain from 0.3976 to 0.9743. Consequently, in the fault diagnosis task on the three-phase flow process, our model surpasses LLMs exceeding 100B parameters, such as GPT-4o, despite having a modest size of 7B parameters. Moreover, our approach outperforms the second-best method by 11.4% in Expression Conciseness, indicating reduced redundancy. Figure 3 compares the average contribution of the top 5 segments in each cases. Our method consistently achieves higher contribution scores than competitive baselines, which confirms the superior conciseness.

Visualization of Explainability: Figure 4 presents a case in *Air Pipeline Blockage Fault* condition comparing captions generated by several competitive methods functioning in the agent workflow. We display only the text segments with high contributions, which are represented by background colors. The captions obtained by DLAC effectively match with K_{task} , exclude irrelevant categories, and diagnose the correct fault category. Conversely, GPT-5.2 and Seed1.6-VL misplace the descriptive focus on irrelevant variables, stemming from untrained and inefficient reflection. GPT-4o exhibits insufficient understanding of the tools, leading to inaccurate selection of descriptive perspectives. Consequently, the contribution scores of their text segments remain low, thereby hindering correct reasoning.

Different Downstream Task Settings: Experimental re-

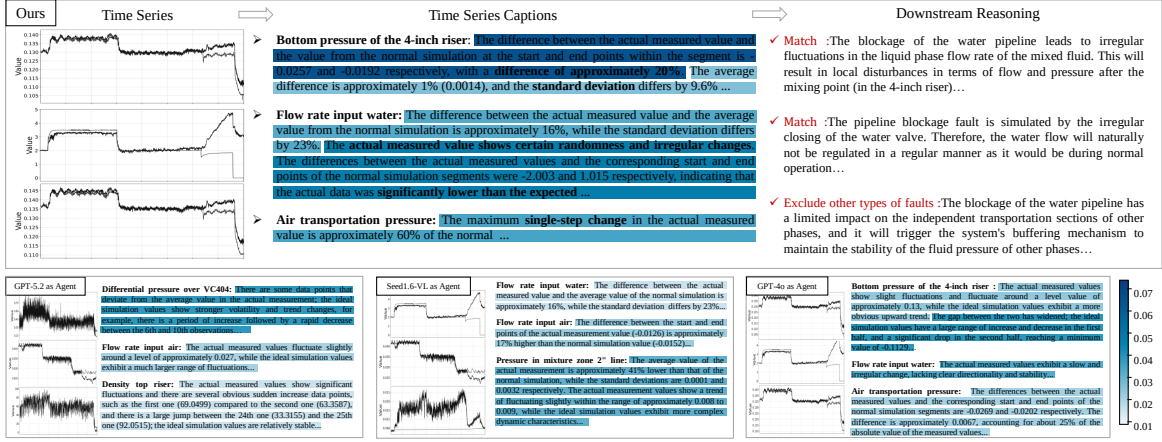


Figure 4. Visualization of explainable time series analysis using captions. **Top**: The proposed method autonomously generates time series captions from data. These captions serve as a semantic foundation for downstream reasoning. This readable process ensures high decision explainability. **Bottom Left, Bottom Middle, and Bottom Right** display three captions generated by different baseline methods.

Table 3. Evaluation of time series captions generated by different baselines in time series forecasting tasks. “Ours(c)” denotes the captions generated by the model trained using a classification-oriented diagnosis task as a proxy. “Ours(f)” represents the captions generated by the model trained using a time series forecasting task as a proxy. “FA” indicates the FActScore. “MAE” denotes the Mean Absolute Error between the predicted values and ground-truth values. Higher values for metrics marked with $\uparrow$ and lower values for metrics marked with $\downarrow$ indicate superior performance.

Method	GPT-4o	Seed1.6-VL	Ours(c)	Ours(f)
FA $\uparrow$	0.711	0.592	0.913	0.902
MAE $\downarrow$	0.61	0.75	0.42	0.39

sults presented in Table 3 demonstrate the following two points. (1) *First, the time series captions generated by our proposed method exhibit cross-task utility.* We apply the time series captions, generated by the model trained on the downstream diagnosis task, to the time series forecasting task. The results show that our approach outperforms competitive baselines by 31.1%. This is because different tasks fundamentally rely on capturing the key dynamics of the time series, differing only in their specific emphasis. Furthermore, the results from Table 3 and Table 1 show that our generated captions achieve higher similarity to human expert annotations than the baselines, indicating genuine semantic quality rather than shortcut optimization for specific tasks. (2) *Second, in the CLFL paradigm, the downstream task can be freely selected as the proxy reward, without affecting the pipeline’s universality.* We add experiments using a forecast regression task as the proxy reward, where ours outperforms baselines. This demonstrates the effectiveness of our pipeline beyond classification-oriented settings. Simultaneously, we evaluate the Spearman’s rank correlation coefficient between the forecasting error and text quality metrics. Experimental results presented in Appendix D.6

Table 4. Evaluation of caption quality and utility on different datasets. “Base Model” denotes the captions generated from the Qwen2.5-VL-7B. “Ours” denotes the captions generated from the TSCAgent optimized by our method from Qwen2.5-VL-7B.

Domain	Metrics	GPT-4o	Base Model	Ours
Three-phase flow	Fault Diagnosis Accuracy	0.8225	0.3976	0.9743
Three-phase flow	Text Quality(FActScore)	0.711	0.081	0.913
Gas turbine	Fault Diagnosis Accuracy	0.5237	0.2314	0.8872
Medical (Fernández et al., 2024)	Disease Diagnosis Accuracy	0.7993	0.4213	0.8977
General (Guan et al., 2026) (Ye et al., 2025)	Classification Accuracy	0.8002	0.4601	0.8343

demonstrate that a smaller forecasting error corresponds to a higher text quality.

Different Domains Settings: We provide other domain scenarios that further demonstrate the generalizability of the proposed method. (1) We applied the agent trained on the three-phase flow process to a real gas turbine power generation scenario, whose background is in Appendix C.2 and the experimental results presented in Appendix D.7. The model achieved a downstream task accuracy of 88.72%, demonstrating the generalization capability of the agent. (2) Moreover, to verify the broad applicability of our approach, we conducted and divided the empirical experiments into two distinct categories: specialized domains and general domains, detailed in Appendix D.8. For the specialized domains, in addition to the aforementioned two industrial scenarios, we introduced a disease diagnosis task from the medical field that focuses on classifying retinal electrical signals (Fernández et al., 2024). For the general domains, we adopted recent open-sourced datasets designed for time series reasoning using LLMs (Guan et al., 2026; Ye et al., 2025). Experimental results presented in Table 4 demonstrate that, on different datasets, the quality of the generated captions has been improved compared to the Base Model,

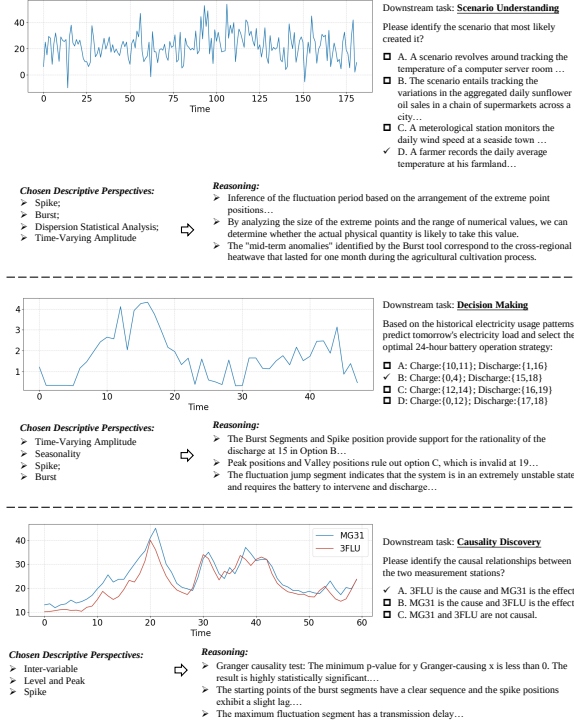


Figure 5. Visualization of captioning and reasoning examples of different downstream tasks in general domains. In each example, the task description, the perspective described in the captions, and the reasoning process based on the caption are all presented.

and more professional time series descriptions can be obtained than those of the general large model GPT-4o. Furthermore, in Figure 5, we present the time-series captioning examples generated by our method on different downstream tasks on general domain datasets. These results verify the broad applicability of our framework in different domain scenarios. Due to space limitations, additional experiments are detailed in Appendix C.3, D.5, D.9, D.10.

5. Community Contribution

After conducting the aforementioned experiments, we sort out the samples yielding correct diagnosis conclusions into a dataset, TFTSC. We incorporated a human verification stage to perform post-hoc sanity checks on TFTSC (See Appendix E.1). The TFTSC dataset consists of over 1800 multivariate time series samples. These samples contain more than 60,000 univariate time series sequences. Each sequence includes caption annotations. The data covers four variable categories collected from eleven sensor channels. Additionally, the dataset spans four system operating states, including 10 fine-grained fault types. The captions in the dataset offer a wide range of descriptive perspectives, achieved through 42 specialized description tools. As illustrated in Figure 20, statistical analysis of the descriptive

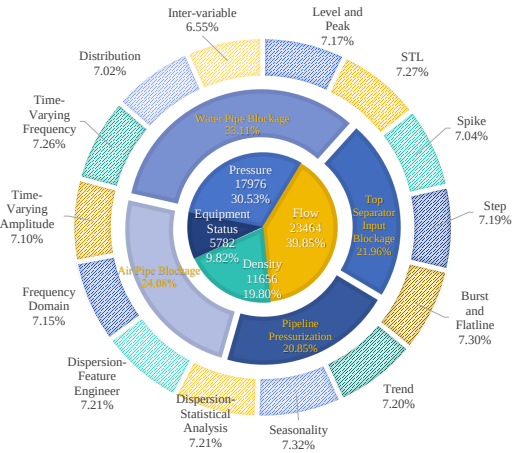


Figure 6. The distribution of data statistics in the TFTSC dataset. The **inner ring** shows variable types. The **middle ring** shows different system operating states. The **outer ring** shows the descriptive perspectives.

perspectives in all generated texts reveals a uniform distribution of invocation frequencies across the 14 categories in the toolset, demonstrating the richness and balance of the dataset. Detailed examples are provided in Appendix E.2.

While contemporary methods have proven effective on general-domain datasets with explicit semantic correlations, TFTSC serves as a novel benchmark with an industrial expert-level descriptive perspective. This benchmark poses a distinct challenge that helps identify the boundaries of current approaches in discerning critical temporal semantics for specialized domains.

6. Conclusion and Future Work

In this paper, we introduce a Caption Label-Free Learning paradigm that circumvents the dependency on ground truth caption annotations. The proposed Dual Loop Agentic Captioning technique enhances both the linguistic quality and professional depth of the generated captions. Empirically, reflective exploration in the inner loop elevates caption quality, as evidenced by a 28.7% increase in the ROUGE-L metric on the GPT-5.2 backbone compared to ablations without the iterative agent workflow. Simultaneously, the RL optimization by proxy reward in the outer loop effectively boosts the fault diagnosis performance of the base model, elevating the accuracy of Qwen2.5-VL-7B from 39.76% to 97.43%. This demonstrates that our proposed exploration-feedback mechanism effectively generates high-quality time series captions from an expert perspective, without any caption label supervision. Furthermore, we release TSTSC, an industrial expert-level dataset, to serve as a benchmark for future research in the community. Future efforts will concentrate on additional human-in-the-loop interaction and verification for risk interception in safety-critical deployment.

Acknowledgements

This work was supported by the grants of the National Natural Science Foundation of China (No. 62125306, No. U25B2050, and No. 62450020) and the State Key Laboratory of Industrial Control Technology, China (No. ICT2026A01).

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Adebayo, J., Gilmer, J., Muelly, M., Goodfellow, I., Hardt, M., and Kim, B. Sanity checks for saliency maps. *Advances in Neural Information Processing Systems*, 31, 2018.
- Ansari, A. F., Heng, A., Lim, A., and Soh, H. Neural continuous-discrete state space models for irregularly-sampled time series. In *International Conference on Machine Learning*, pp. 926–951. PMLR, 2023.
- Arrieta, A. B., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., García, S., Gil-López, S., Molina, D., Benjamins, R., et al. Explainable artificial intelligence (xai): Concepts, taxonomies, opportunities and challenges toward responsible ai. *Information fusion*, 58:82–115, 2020.
- Belkhouja, T., Yan, Y., and Doppa, J. R. Dynamic time warping based adversarial framework for time-series domain. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(6):7353–7366, 2022.
- Bello, M., Amador, R., García, M.-M., Del Ser, J., Mesejo, P., and Cordon, Ó. The level of strength of an explanation: A quantitative evaluation technique for post-hoc xai methods. *Pattern Recognition*, 161:111221, 2025.
- Chai, Z. and Zhao, C. A fine-grained adversarial network method for cross-domain industrial fault diagnosis. *IEEE Transactions on Automation Science and Engineering*, 17(3):1432–1442, 2020.
- Feng, L. and Zhao, C. Fault description based attribute transfer for zero-sample industrial fault diagnosis. *IEEE Transactions on Industrial Informatics*, 17(3):1852–1862, 2020.
- Fernández, I., Cuadrado-Asensio, R., Larriba, Y., Rueda, C., and Coco-Martín, R. M. A comprehensive dataset of pattern electroretinograms for ocular electrophysiology research. *Scientific Data*, 11(1):1013, 2024.
- Fons, E., Kaur, R., Zeng, Z., Palande, S., Balch, T., Vyetrenko, S., and Veloso, M. Tadacap: Time-series adaptive domain-aware captioning. In *Proceedings of the 5th ACM International Conference on AI in Finance*, pp. 54–62, 2024.
- Gao, S., Koker, T., Queen, O., Hartvigsen, T., Tsiligkaridis, T., and Zitnik, M. Units: A unified multi-task time series model. *Advances in Neural Information Processing Systems*, 37:140589–140631, 2024.
- Gemini Team, Google DeepMind. Gemini 3 technical report. Technical report, Google DeepMind, 11 2025. URL <https://deepmind.google/models/gemini/>. Accessed: 2025-12-27.
- Guan, T., Meng, Z., Li, D., Wang, S., Yang, C.-H. H., Wen, Q., Liu, Z., Siniscalchi, S. M., Jin, M., and Pan, S. Timeomni-1: Incentivizing complex reasoning with time series in large language models. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=kOIclg7muL>.
- Guo, D., Wu, F., Zhu, F., Leng, F., Shi, G., Chen, H., Fan, H., Wang, J., Jiang, J., Wang, J., et al. Seed1. 5-vl technical report. *arXiv preprint arXiv:2505.07062*, 2025.
- He, J., Li, K., Zang, Y., Fu, H., Fu, Q., Xing, J., and Cheng, J. Efficient multi-task reinforcement learning with cross-task policy guidance. *Advances in Neural Information Processing Systems*, 37:117997–118024, 2024.
- Hong, W., Yu, W., Gu, X., Wang, G., Gan, G., Tang, H., Cheng, J., Qi, J., Ji, J., Pan, L., et al. Glm-4.1 v-thinking: Towards versatile multimodal reasoning with scalable reinforcement learning. *arXiv e-prints*, pp. arXiv–2507, 2025.
- Howard, J. and Ruder, S. Universal language model fine-tuning for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 328–339, 2018.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al. Lora: Low-rank adaptation of large language models. *International Conference on Learning Representations*, 1(2):3, 2022.
- Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2):1–55, 2025.

- Ismail, A. A., Gunady, M., Corrada Bravo, H., and Feizi, S. Benchmarking deep learning interpretability in time series predictions. *Advances in Neural Information Processing Systems*, 33:6441–6452, 2020.
- Jesson, A., Beltran-Velez, N., Chu, Q., Karlekar, S., Kossen, J., Gal, Y., Cunningham, J. P., and Blei, D. Estimating the hallucination rate of generative ai. *Advances in Neural Information Processing Systems*, 37:31154–31201, 2024.
- Jhamtani, H. and Berg-Kirkpatrick, T. Truth-conditional captioning of time series data. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pp. 719–733, 2021.
- Jiang, Y., Chen, Y., Li, X., Chao, Q., Liu, S., and Cong, G. Fstllm: Spatio-temporal llm for few shot time series forecasting. In *International Conference on Machine Learning*, pp. 27492–27509. PMLR, 2025.
- Kim, D., Hong, M., Park, J., and Oh, S. Conflict-averse gradient aggregation for constrained multi-objective reinforcement learning. In *International Conference on Learning Representations*, volume 2025, pp. 36498–36527, 2025.
- Kister, H. Z. *Distillation troubleshooting*. John Wiley & Sons, 2011.
- Kong, Y., Yang, Y., Hwang, Y., Du, W., Zohren, S., Wang, Z., Jin, M., and Wen, Q. Time-mqa: Time series multi-task question answering with context enhancement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pp. 29736–29753, 2025.
- Lei, P., Song, J., Hao, Y., Chen, T., Zhang, Y., JIA, L., Li, Y., et al. Itformer: Bridging time series and natural language for multi-modal qa with large-scale multitask dataset. In *International Conference on Machine Learning*, pp. 63324–63344. PMLR, 2025.
- Li, B. and Zhao, C. S2s-fdd: Bridging industrial time series and natural language for explainable zero-shot fault diagnosis. In *2025 CAA Symposium on Fault Detection, Supervision, and Safety for Technical Processes (SAFEPROCESS)*, pp. 1–6, 2025. doi: 10.1109/SAFEPROCESS67117.2025.11268252.
- Li, H., Chen, X., Xu, Z., Li, D., Hu, N., Teng, F., Li, Y., Qiu, L., Zhang, C. J., Qing, L., et al. Exposing numeracy gaps: A benchmark to evaluate fundamental numerical abilities in large language models. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 20004–20026, 2025.
- Li, S., Wang, L., Zhang, R., Chang, X., Liu, X., Xie, Y., Qi, Y., and Song, L. Temporal logic point processes. In *International Conference on Machine Learning*, pp. 5990–6000. PMLR, 2020.
- Li, T., Ma, M., and Peng, X. Beyond accuracy: ensuring correct predictions with correct rationales. *Advances in Neural Information Processing Systems*, 37:43164–43188, 2024.
- Lin, C.-Y. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*, pp. 74–81, 2004.
- Lin, H., Tan, C., Wu, L., Liu, Z., Gao, Z., and Li, S. Z. An extensive survey with empirical studies on deep temporal point process. *IEEE Transactions on Knowledge and Data Engineering*, 37(4):1599–1619, 2024.
- Liu, A., Mei, A., Lin, B., Xue, B., Wang, B., Xu, B., Wu, B., Zhang, B., Lin, C., Dong, C., et al. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.
- Liu, H., Kamarthi, H., Kong, L., Zhao, Z., Zhang, C., and Prakash, B. A. Time-series forecasting for out-of-distribution generalization using invariant learning. In *International Conference on Machine Learning*, pp. 31312–31325. PMLR, 2024.
- Miller, T. Explanation in artificial intelligence: Insights from the social sciences. *Artificial intelligence*, 267:1–38, 2019.
- Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W.-t., Koh, P., Iyyer, M., Zettlemoyer, L., and Hajishirzi, H. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In Bouamor, H., Pino, J., and Bali, K. (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 12076–12100, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.741. URL <https://aclanthology.org/2023.emnlp-main.741/>.
- Moor, M., Banerjee, O., Abad, Z. S. H., Krumholz, H. M., Leskovec, J., Topol, E. J., and Rajpurkar, P. Foundation models for generalist medical artificial intelligence. *Nature*, 616(7956):259–265, 2023.
- OpenAI. Chatgpt (gpt-5.2). <https://chat.openai.com/>, 2025. Large language model.
- Petruzzelli, A., Musto, C., Laraspata, L., Rinaldi, I., de Gemmis, M., Lops, P., and Semeraro, G. Instructing and prompting large language models for explainable cross-domain recommendations. In *Proceedings of the 18th ACM Conference on Recommender Systems*, pp. 298–308, 2024.

- Raghu, A., Chandak, P., Alam, R., Gutttag, J., and Stultz, C. Sequential multi-dimensional self-supervised learning for clinical time series. In *International Conference on Machine Learning*, pp. 28531–28548. PMLR, 2023.
- Rao, C. R. et al. Information and the accuracy attainable in the estimation of statistical parameters. *Bull. Calcutta Math. Soc.*, 37(3):81–91, 1945.
- Ren, L., Wang, H., Li, J., Tang, Y., and Yang, C. Aigc for industrial time series: From deep-generative models to large-generative models. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 55:7774–7791, 2025.
- Ruiz-Cárcel, C., Cao, Y., Mba, D., Lao, L., and Samuel, R. Statistical process monitoring of a multiphase flow facility. *Control Engineering Practice*, 42:74–88, 2015.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y., Wu, Y., et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Shi, X., Xue, S., Wang, K., Zhou, F., Zhang, J., Zhou, J., Tan, C., and Mei, H. Language models can improve event prediction by few-shot abductive reasoning. *Advances in Neural Information Processing Systems*, 36:29532–29557, 2023.
- Sivill, T. and Flach, P. Limesegment: Meaningful, realistic time series explanations. In *International Conference on Artificial Intelligence and Statistics*, pp. 3418–3433. PMLR, 2022.
- Slack, D., Krishna, S., Lakkaraju, H., and Singh, S. Explaining machine learning models with interactive natural language conversations using talktomodel. *Nature Machine Intelligence*, 5(8):873–883, 2023.
- Sowdaboina, P. K. V., Chakraborti, S., and Sripada, S. Learning to summarize time series data. In *International Conference on Intelligent Text Processing and Computational Linguistics*, pp. 515–528. Springer, 2014.
- Sundararajan, M., Taly, A., and Yan, Q. Gradients of counterfactuals. *arXiv preprint arXiv:1611.02639*, 2016.
- Sweller, J. Cognitive load during problem solving: Effects on learning. *Cognitive science*, 12(2):257–285, 1988.
- Tan, M., Merrill, M., Gupta, V., Althoff, T., and Hartvigsen, T. Are language models actually useful for time series forecasting? *Advances in Neural Information Processing Systems*, 37:60162–60191, 2024.
- Team, Q. Qwen2.5-vl, January 2025a. URL <https://qwenlm.github.io/blog/qwen2.5-vl/>.
- Team, Q. Qwen3 technical report, 2025b. URL <https://arxiv.org/abs/2505.09388>.
- Trabelsi, M., Boyd, A., Cao, J., and Uzunalioglu, H. Time series language model for descriptive caption generation. *arXiv preprint arXiv:2501.01832*, 2025.
- Troncoso-García, Á. R., Martínez-Ballesteros, M., Martínez-Álvarez, F., and Troncoso, A. A new metric based on association rules to assess feature-attribution explainability techniques for time series forecasting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(5):4140–4155, 2025.
- Vielhaben, J., Lapuschkin, S., Montavon, G., and Samek, W. Explainable ai for time series via virtual inspection layers. *Pattern Recognition*, 150:110309, 2024.
- von Werra, L., Belkada, Y., Tunstall, L., Beeching, E., Thrush, T., Lambert, N., Huang, S., Rasul, K., and Gallouédec, Q. Trl: Transformer reinforcement learning. <https://github.com/huggingface/trl>, 2020.
- Wang, J., Li, R., Li, R., Fu, B., and Chen, D. Z. Hmck-rautoencoder: An interpretable deep learning framework for time series analysis. *IEEE Transactions on Emerging Topics in Computing*, 10(1):99–111, 2022.
- Wang, J., Sun, K., Luo, L., Wei, W., Hu, Y., Liew, A. W., Pan, S., and Yin, B. Large language models-guided dynamic adaptation for temporal knowledge graph reasoning. *Advances in Neural Information Processing Systems*, 37:8384–8410, 2024a.
- Wang, Y., Wu, H., Dong, J., Qin, G., Zhang, H., Liu, Y., Qiu, Y., Wang, J., and Long, M. Timexer: Empowering transformers for time series forecasting with exogenous variables. *Advances in Neural Information Processing Systems*, 37:469–498, 2024b.
- Xie, Z., Li, Z., He, X., Xu, L., Wen, X., Zhang, T., Chen, J., Shi, R., and Pei, D. Chatts: Aligning time series with llms via synthetic data for enhanced understanding and reasoning. In *Proceedings of the VLDB Endowment*, 2025.
- Yang, H., Liu, X.-Y., and Wang, C. D. Fingpt: Open-source financial large language models. *FinLLM Symposium at IJCAI*, 2023.
- Yang, J., Zhao, C., Guay, M., and Cao, Z. Time-series2report prompting enables adaptive large language model management of lithium-ion batteries. *arXiv preprint arXiv:2512.16453*, 2025.
- Ye, W., Liu, J., Cao, D., Yang, W., and Liu, Y. When llm meets time series: Can llms perform multi-step

time series reasoning and inference. *arXiv preprint arXiv:2509.01822*, 2025.

Yu, J., Hunter, J., Reiter, E., and Sripada, S. Recognising visual patterns to communicate gas turbine time-series data. In *Applications and Innovations in Intelligent Systems X: Proceedings of ES2002, the Twenty-second SGAI International Conference on Knowledge Based Systems and Applied Artificial Intelligence*, pp. 105–118. Springer, 2003.

Yuan, H., Yu, H., Gui, S., and Ji, S. Explainability in graph neural networks: A taxonomic survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(5):5782–5799, 2022.

Zhang, M., Qian, F., and Liu, Q. Learning task belief similarity with latent dynamics for meta-reinforcement learning. In *The Thirteenth International Conference on Learning Representations*, 2025.

Zhang*, T., Kishore*, V., Wu*, F., Weinberger, K. Q., and Artzi, Y. Bertscore: Evaluating text generation with bert. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=SkeHuCVFDr>.

Zhang, X., Li, M., and Wu, J. Co-occurrence is not factual association in language models. *Advances in Neural Information Processing Systems*, 37:64889–64914, 2024.

Zhang, Y., Sun, Q., Qi, D., Liu, J., Ma, R., and Petrosian, O. Shaptime: A general xai approach for explainable time series forecasting. In *Proceedings of SAI Intelligent Systems Conference*, pp. 659–673. Springer, 2023.

Zhao, J., Yue, J., Zhao, C., and Chen, C. Adjust to reality: Llm-driven test-time semantic adjustment for zero-shot fault diagnosis. *Control Engineering Practice*, 164: 106406, 2025a.

Zhao, J., Zhao, C., and Yue, J. Align knowledge with time-series: Cross-modal domain knowledge activation for llm-enabled zero-shot fault diagnosis. *Journal of Process Control*, 155:103534, 2025b.

Zheng, J., Chen, X., and Zhao, C. Interval-aware probabilistic slow feature analysis for irregular dynamic process monitoring with missing data. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 53(10):6553–6564, 2023.

Zhou, L., Yashwante, P., Fisher, M., Sampieri, A., Zhou, Z., Galasso, F., and Yu, R. Cats-bench: Can language models describe numeric time series? *arXiv preprint arXiv:2509.20823*, 2025.

A. Supplementary Details of Time Series Captioning

A.1. Details of Specialized Captioning Toolset

Table 5. Details of descriptive tools. The toolset comprises 14 categories and 42 distinct tool types (Columns 1 and 2), encompassing diverse perspectives such as anomalies, structural breaks, trend, seasonality, fluctuation, and the relationship between variables. Column 3 provides partial instructions used for the numerical parsing of computational results.

Category	Tools	Instruction
Level and Peak	Mean; Median; 90th percentile	Determine the overall numerical level and depict the peak behavior.
STL	STL	Be capable of flexibly responding to asymmetric and unstable seasonal fluctuations. Among the three factors - trend, seasonal, and residual, the one with the largest variance is the most significant. The larger the Mean of the Trend component - Mean, the higher the overall value; the larger the Standard deviation of the Trend component - Standard deviation, the more obvious the systematic changes such as increase, decrease, or drift, and the smaller it is, the more stable it is; the larger the Standard deviation of the Seasonal component - Standard deviation, the stronger the periodicity; the larger the Standard deviation of the Residual component - Standard deviation, the stronger the randomness and the more outliers.
Spike	Percentage of 3σ outliers (%); Spike existence; Spike count & Spike position; Max spike amplitude; Mean spike amplitude	Percentage of 3σ outliers (%): What is the number of suspected outliers such as sudden spikes; Spike count&Spike position: number and position of spikes; Max spike amplitude (residue): The impact strength of the most severe spike; Mean spike amplitude: average peak impact strength. If the value of Spike existence is 0, then after detrending and detrending of the cycle, no obvious sudden spikes are found; if it is 1, then there has been at least one significant sudden abnormality.
Step	Step detection, step position, detection, duration	Step detection: The instantaneous jump is large enough (greater than 2.5 times the standard deviation) and the means before and after the jump are significantly different (the difference is greater than 2.5 times the standard deviation), and the new level is maintained for N consecutive points after the jump. If the "step existence" is 1, then there is a clear step.
Burst and Flatline	Burst existence; Burst count; Burst segments; Flatline existence; Flatline count; Flatline segments	Detect two types of complex abnormal patterns in time series: continuous bursts (Burst, which refers to a cluster of sharp peaks occurring densely within a short period of time) and long-term stagnation (Flatline, which refers to a long period of stable values with minimal changes).
Trend	Start-end difference; Maximum single-step change; Trend analysis	Check whether the sequence of detection times shows any obvious increase, decrease, or fluctuation around a certain level, and also determine whether there are frequent retracements or step-like trends.
Seasonality	Autocorrelation; Form factor	The closer "Autocorrelation" is to 1, the more likely it is that this lag step can be regarded as a strong candidate for a cycle; If the autocorrelation of all lags is close to 0, then there is almost no periodic structure (random or high noise); The closer the "form factor" is to 1.11, the closer the shape is to a sine wave. The larger the value is, the closer it is to an impulse signal.
Dispersion-Statistical Analysis	Coefficient of variation; Range; Maximum single-step change; Maximum value; Minimum value	Coefficient of variation: Standard deviation that can be compared across scales; IQR: The width of the middle 50% of the data is a robust dispersion that is not affected by extreme values.
Dispersion-Feature Engineer	Mean signal amplitude; Root Mean Square (RMS); Form factor; Peak factor; SNR (linear); SNR (dB)	Mean signal amplitude: The average strength of the signal, reflecting the overall vibration intensity; Root Mean Square: Energy intensity, reflecting overall vibration intensity with a greater emphasis on large amplitude values The smaller the "peak factor" is, the closer it is to a smooth waveform; "SNR (linear)" represents the ratio of signal energy to noise energy. The larger the value, the less noise there is, and the more valuable it is for observation; "SNR (dB)" within the range of 10 to 20 falls within the acceptable range for normal engineering signals. The smaller the value, the more noise there is, and this reduces the value of the signal for observation.
Frequency Domain	Centroid frequency; Peak frequency; Frequency skewness; Frequency kurtosis	Centroid frequency: Weighted average frequency in the frequency domain; Peak frequency: Main cycle; Frequency skewness: Whether the distribution of spectral energy is symmetrical; Frequency kurtosis: Is the spectrum "sharp and concentrated". A high centroid frequency indicates rapid oscillation and high-frequency noise, while a low centroid frequency suggests slow fluctuation; If the frequency skewness is greater than zero, it indicates that the low frequencies are dominant, but there are also high-frequency fluctuations; if it is less than zero, it means that the high frequencies are dominant, but there are also slow components; The smaller the frequency kurtosis is, the more noise components there are.
Time-Varying Amplitude	Sliding window size; Sliding STD min; Sliding STD max; Maximum sliding STD jump; Max fluctuation segment; Fluctuation jump segment	Time varying analysis of fluctuation amplitude is used to characterize the stability and operational status of a system. The "Sliding window size" refers to the duration of each assessment of "local fluctuations"; "Sliding STD min/max" represents the fluctuation intensity of the most stable and the most intense sections in the sequence; The larger the "Sliding STD max/min ratio", the greater the variation in fluctuation amplitude; "Max fluctuation segment" is the time interval with the greatest fluctuation amplitude; "Fluctuation jump segment" is the time position where the fluctuation suddenly changes.
Time-Varying Frequency	Frequency trend; Significant frequency change points	The time-varying analysis of fluctuation frequency is used to characterize the changes in the dynamic mechanism of a system. Frequency trend indicates whether the frequency is drifting from low frequency to high frequency or from high frequency to low frequency.
Distribution	Interquartile range (IQR); 25th percentile; 50th percentile (Median); 75th percentile; Skewness; Kurtosis	Compared to the Gaussian distribution, the skewness of the Gaussian distribution is 0. If the skewness is greater than 0, it indicates that the time series data shows sporadic peaks, (a right skew with a long right tail, indicating more abnormal fluctuations towards larger values). If the skewness is less than 0, it indicates that the time series data shows an occasional sharp drop (a left skew with a long left tail, indicating more abnormal fluctuations towards smaller values). Compared to the Gaussian distribution, the kurtosis of the Gaussian distribution is 3.5. If the kurtosis is greater than 3, it indicates a heavy-tailed distribution, exhibiting high peaks and heavy tails, with many extreme outliers; if the kurtosis is less than 3, it indicates a flat distribution, exhibiting low peaks and light tails, with more uniform data.
Inter-variable	Sequence covariance; Cross-correlation peak value; Lag corresponding to cross-correlation peak; Minimum p-value for x; Granger-causing y and y; Granger-causing x	If the sequence covariance is positive, it indicates that they rise and fall together. If it is negative, it indicates that one rises while the other falls. If the absolute value is close to 0, it indicates a weak linear relationship. The closer the cross-correlation peak value is to 1, the stronger the time-dependent relationship. If the Minimum p-value for x Granger-causing y is smaller (less than 0.05), the Granger causality between x and y is more significant; if the Minimum p-value for y Granger-causing x is smaller (less than 0.05), the Granger causality between y and x is more significant; if both directions are small, the system may have feedback.

A.2. Calculation of Minimum Semantic Length

We define the minimum semantic length as a guarantee of semantic validity, denoted as L_{min} , which is the minimal duration required for a time series segment to encapsulate independent semantics. While appropriate prior knowledge in specific domains may directly determine this length, we design a dynamic computation method applicable across diverse domains.

Specifically, We design a method to calculate L_{min} governed by physical and information dual constraints. From a physical perspective, the sampling window length must suffice to encompass the dynamic memory length of the system, thereby suppressing transient noise. From an information perspective, the saturation of the marginal information gain resulting from increased window length indicates that the window captures complete information regarding the local steady state.

For a given raw time series stream $X = \{x_1, x_2, \dots, x_N\}$, we seek the minimum forward window length L_{min} at an arbitrary time step t .

The Physical Bound: We determine the system decorrelation time τ via the autocorrelation function R . The physical decorrelation time τ_{phy} is defined as the first instance where $R(\tau)$ decays to $1/e$:

$$\tau_{phy} = \min\{\tau \mid |R(\tau)| \leq 1/e\} \quad (8)$$

The Information Bound: We utilize spectral entropy to measure the signal complexity within the window. Short windows are primarily dominated by high-frequency noise; however, as the window extends and structured semantics (such as periodicity and trend) emerge, the entropy value undergoes a transition from rapid variation to stabilization. For a candidate window $X_L = X[t : t + L]$ of length L , the spectral entropy $H_{se}(L)$ is defined as:

$$H_{se}(w) = - \sum_k p_k(w) \log p_k(w) \quad (9)$$

We define the rate of change of spectral entropy with respect to window growth as the Marginal Information Gain (MIG):

$$MIG(L) = \left| \frac{\partial H_{se}(L)}{\partial L} \right| \quad (10)$$

The dual physical and information constraints are formulated as follows:

$$\begin{aligned} L_{min} &= \min \{L \in \mathbb{Z}^+\} \\ \text{s.t. } L &> \alpha \cdot \tau_{phy} \\ MIG(L) &< \epsilon \end{aligned} \quad (11)$$

where α represents a preset safety coefficient and ϵ denotes a preset convergence threshold.

A.3. Calculation of Text Segment Contribution

Integrated Gradients (IG) is a commonly used gradient-based attribution method, which estimates the contribution of input features to model output by interpolating between a reference input and the real input and integrating gradients (Sundararajan et al., 2016). In the time series description task, we are more concerned about the contribution of text segments rather than performing a token-by-token calculation. Supposed that the time series description o is composed of the content returned by the execution of M tool call instructions $C = \{c_1, c_2, \dots, c_M\}$, $Con(o)$ considers the contribution of the description o_i on each time series symptom to the model output. Specifically, each instruction c_i is executed and parsed to obtain a time series symptom description o_i :

$$o_i = \mathcal{P}(\mathcal{E}(c_i)) \quad (12)$$

$$o = \oplus_{i=1}^M o_i \quad (13)$$

The text segments range of o_i in o is denoted as:

$$St_i = [it_i^{\text{start}}, it_i^{\text{end}}] \quad (14)$$

Suppose that the overall time series description o can be tokenized into a token sequence with the length of T . The vector representation of the token sequence is obtained through the word embedding layer:

$$o = (o^1, o^2, \dots, o^T) \quad (15)$$

$$E(o) = (e_1, e_2, \dots, e_T), e_i \in \mathbb{R}^d \quad (16)$$

where d is the word vector dimension. The all-zero sequence embedding is selected as the reference sequence, denoted as $E(o')$:

$$E(o') = (e_0, e_0, \dots, e_0) \quad (17)$$

where e_0 is an all-zero embedding vector. The interpolation path from the reference sequence to the real sequence is defined as:

$$E^{(\gamma)}(o) = E(o') + \gamma \cdot (E(o) - E(o')), \gamma \in [0, 1] \quad (18)$$

The embedding vector of the i -th token on the interpolation path is recorded as $e_i^{(\gamma)}$:

$$e_i^{(\gamma)} = e_0 + \gamma \cdot (e_i - e_0) \quad (19)$$

The logits matrix obtained by LLM $\mathcal{H}$ forward reasoning on the word embedding $E(o)$ in π can be denoted as $\mathbf{F}_{\mathcal{H}}(E(o)) \in \mathbb{R}^{T \times V}$, where T is the output sequence length, and V is the vocabulary size. The prompt p_d contains the requirement that the fault type number should be output at the position of the last token. Then the last token (fault type serial number) of the output can be recorded as:

$$f_{\mathcal{H}}(E(o)) = \max_{k \in \{1, 2, \dots, V\}} \mathbf{F}_{\mathcal{H}}(E(o))_{-1, k} \quad (20)$$

Then the Integrated Gradients of the i -th token is defined as:

$$IG_i(o) = (e_i - e_0) \cdot \int_0^1 \frac{\partial f_{\mathcal{H}}(E^{(\gamma)}(o))}{\partial e_i^{(\gamma)}} d\gamma \quad (21)$$

In numerical implementation, the Equation 21 is approximated by l interpolation steps:

$$IG_i(o) \approx (e_i - e_0) \cdot \frac{1}{l} \sum_{k=1}^l \frac{\partial f_{\mathcal{H}}(E^{(\frac{k}{l})}(o))}{\partial e_i^{(\frac{k}{l})}} \quad (22)$$

The importance score of the i -th token is calculated by the vector norm as follows:

$$s_i = \|IG_i(o)\|_2, i = 1, \dots, T \quad (23)$$

Through the offset mapping of $\mathcal{H}$'s tokenizer, each token is mapped back to the text's character range. The calculation method for the contribution $\text{Con}(o_j)$ of the j -th time series symptom description o_j is:

$$\text{Con}(o_j) = \text{mean}(\{s_i \mid \text{offset}(o^i) \cap St_j \neq \emptyset\}) \quad (24)$$

The reward encourages the agent to generate time series descriptions that are as highly contributive as possible to downstream utilities. Specifically, calculate the minimum value of the contributions of each time series symptom description o_i :

$$\text{Con}_{\min} = \min_{i=1, \dots, M} \text{Con}(o_i) \quad (25)$$

Since the calculation of Integrated Gradients uses the L2 norm, there is theoretically no strict upper bound, and outliers may appear in practice. Therefore, percentile normalization is performed, which is less sensitive to extreme values and more robust. Specifically, during training, we collect N_Q $\text{Con}_{\min}$ from several adjacent batches and sort them:

$$\text{Con}_{\min(1)} \leq \text{Con}_{\min(2)} \leq \dots \leq \text{Con}_{\min(N_Q)} \quad (26)$$

For the sorted calculation of percentiles, perform percentile normalization on $Con_{\min}$ and truncate values exceeding $[0, 1]$:

$$\tilde{R}_Q = \frac{Con_{\min} - Q_{0.05}}{Q_{0.95} - Q_{0.05}} \quad (27)$$

$$\tilde{Con}_{\min} = \begin{cases} \tilde{R}_Q & \text{if } 0 < \tilde{R}_Q < 1 \\ 1 & \text{if } \tilde{R}_Q > 1 \\ 0 & \text{if } \tilde{R}_Q < 0 \end{cases} \quad (28)$$

where $Q_{0.05} = R_{\text{scr}([0.05 \cdot N_Q])}$, $Q_{0.95} = R_{\text{scr}([0.95 \cdot N_Q])}$. Here, $\tilde{R}_Q$ is the result of percentile normalization for $Con_{\min}$.

A.4. Extended Operations Regarding Industrial Scenario

In industrial operation and maintenance scenarios, anomaly detection typically serves as a preliminary stage prior to fault diagnosis. This process involves modeling historical time series data characterizing normal operations to reconstruct online data, subsequently identifying anomalies based on reconstruction errors. Therefore, in the context of industrial applications, the proposed DLAC framework includes two optional extended operations in addition to the core components described in the main text.

A.4.1. SELECTION OF KEY VARIABLES

Larger discrepancies between actual measurements and reconstructed data indicate a higher probability of anomalies and are therefore more worthy of being captioned. Methodologically, we compute the reconstruction error for each variable at every time step to generate a reconstruction error sequence for each variable. The mean and variance of these reconstruction error sequences are aggregated to form a meta difference table, which serves as input for the TSCAgent to perform variable selection. Notably, the mean and variance statistics are standardized to enable direct comparison across variables. A higher variance implies a more abrupt divergence between the observed temporal pattern of the variable and the normal temporal pattern. Conversely, the discrepancy may manifest as a persistent deviation that remains relatively invariant over time. We design prompts instructing the TSCAgent to select the top-ranking variables based on their reconstruction errors and utilize them for operation and maintenance analysis and decision making, such as diagnosing potential system malfunctions.

A.4.2. REDUNDANCY REMOVAL

We incorporate reconstructed data to enhance the caption o_{final} . Specifically, in addition to generating captions o_{final} for the actual measured time series, we utilize the reconstructed time series as a normal reference to produce corresponding captions $\hat{o}_{\text{final}}$. By comparing these two sequences, we identify and eliminate segments with minimal discrepancies as redundant descriptions to obtain a more refined caption text.

A.5. Further Discussion on the CLFL

A.5.1. LIMITATIONS & FUTURE EXTENSIONS

Our proposed framework establishes an effective foundation for time series captioning. However, we outline several directions to further expand its applicability. First, to ensure reliable and unambiguous feedback, the reward in CLFL is directly derived from downstream ground-truth labels rather than arbitrary heuristic metrics. For extreme scenarios where downstream labels are entirely unavailable, exploring indirect reward mechanisms based on text coherence and information gain presents an interesting future direction. Furthermore, our experiments clearly demonstrate the strong cross-task utility of the generated captions. However, when there are multiple downstream requirements, especially when multiple downstream tasks conflict with each other, a single reward signal is insufficient. Extending our approach to multi-objective reinforcement learning, using mature techniques such as weight allocation (Kim et al., 2025) or alternating training (He et al., 2024), would be a natural progression to balance these conflicts.

Beyond the reward design, our primary focus is on establishing the core CLFL paradigm rather than specifically targeting out-of-distribution domain generalization. Future work can seamlessly adopt meta-RL (Zhang et al., 2025) during training or apply cross-domain fine-tuning from the source model to handle distinct domain shifts. Finally, DLAC’s tool execution greatly reduces hallucinations, yet errors may still occur during numerical parsing. Confidence calibration can be incorporated in future work to provide stronger safety guarantees.

A.5.2. RELEVANCE TO THE FIELD OF TEMPORAL POINT PROCESS

Temporal Point Processes (TPPs) focus on representing and modeling event sequences in continuous time to predict the probability of future events or discover the underlying causal relationships (Lin et al., 2024). LLM-based TPPs (Shi et al., 2023) and Temporal Logic Point Processes (TLPPs) (Li et al., 2020) enhance semantic processing and interpretability. Semantics extracted by TSCAgent are equivalent to the events in TPPs. Logical rules for event semantic parsing in LLM-based TPPs and TLPPs are beneficial for downstream reasoning based on time-series captions. The time-series captioning task serves as an extension of event modeling in TPPs, providing more intuitive interpretability. Additionally, exhaustively defining rigid logical rules for TLPPs is highly challenging in complex industrial scenarios. In contrast, our framework leverages LLMs for implicit logical reasoning over time-series fluctuations, offering superior flexibility for open-domain problems.

B. Proof of Semantic Coverage Assurance Scheme

B.1. Proof of Remark 3.3

Proof Regarding the partition of evolutionary stages

$$\Lambda(T) = \sum_{k=1}^n l_k \quad (29)$$

The total semantic omission of the process is defined as

$$\mathcal{E} = \sum_{k=1}^n E_k = \sum_{k=1}^n \int_{t_{k-1}}^{t_k} \text{dist}_{\mathcal{M}}^2(P(t), d_k) dt \quad (30)$$

Local Tangent Space Approximation: Since the X length is sufficiently small relative to the total evolution, we approximate the local manifold neighborhood structure using its tangent space $T_{d_k} \mathcal{M}$ at the centroid d_k . This process corresponds to a local linearization of the manifold $T_{d_k} \mathcal{M}$ represents the tangent space at a certain point d_k on the manifold $\mathcal{M}$.

Arc-Length Parameterization: Let s denote the arc length parameter along the curve, such that the centroid corresponds to $s = 0$. The measure of information loss is performed in the intrinsic geometry of the curve. Under the local linear approximation, the Riemannian distance between a point $P(s)$ on the curve and the centroid d_k is approximated by the arc length difference:

$$\text{dist}_{\mathcal{M}}(P(s), d_k) \approx |s| \quad (31)$$

Consequently, the semantic omission E_k for the stage with arc length l_k (interval $[-l_k/2, l_k/2]$) is calculated as:

$$E_k = \int_{-l_k/2}^{l_k/2} s^2 ds = \left[\frac{s^3}{3} \right]_{-l_k/2}^{l_k/2} = \frac{l_k^3}{12} \quad (32)$$

This coefficient $\frac{1}{12}$ represents the fundamental geometric constant for the variance of a uniform distribution on a linear segment.

Consider the constrained optimization problem:

$$\begin{aligned} \min_{\{l_k\}} \mathcal{J} &= \sum_{k=1}^n \frac{l_k^3}{12} \\ \text{s.t. } &\sum_{k=1}^n l_k = \Lambda(T), \quad l_k > 0 \end{aligned} \quad (33)$$

By introducing the Lagrange multiplier λ , we construct the Lagrangian function $\mathcal{L}(l_1, \dots, l_n, \lambda)$:

$$\mathcal{L} = \sum_{k=1}^n \frac{l_k^3}{12} + \lambda \left(\Lambda(T) - \sum_{k=1}^n l_k \right) \quad (34)$$

Computing the partial derivative with respect to the k th length variable l_k :

$$\frac{\partial \mathcal{L}}{\partial l_k} = \frac{d}{dl_k} \left(\frac{l_k^3}{12} \right) - \lambda = \frac{3l_k^2}{12} - \lambda = \frac{l_k^2}{4} - \lambda \quad (35)$$

Setting $\frac{\partial \mathcal{L}}{\partial l_k} = 0$ yields

$$\frac{l_k^2}{4} = \lambda \implies l_k^2 = 4\lambda \implies l_k = 2\sqrt{\lambda} \quad (36)$$

For all $k = 1, \dots, n$, the optimal l_k must equal the constant $2\sqrt{\lambda}$. This implies that the lengths of all stages are identical in the optimal solution, specifically

$$l_k = \frac{\Lambda(T)}{n} \quad (37)$$

Furthermore, since E_k is a strictly convex function with respect to l_k , the identified stationary point corresponds to the global minimum. Consequently, substituting into Equation 30 and 32, we derive the ideal minimum semantic omission $\mathcal{E}$

$$\mathcal{E}_{\text{opt}} = \frac{1}{12} \frac{\Lambda(T)^3}{n^2} \quad (38)$$

The finite upper bound of $\Lambda(T)$ is theoretically guaranteed by the Lipschitz continuity of the encoder Φ over the finite time horizon $[0, T]$. Specifically, since Φ is parameterized by finite weights and composed of bounded activation functions (typical of deep neural networks like Transformers), it enforces a constraint on the instantaneous rate of semantic change, denoted as $v(t) \leq K$, where K is the Lipschitz constant. Consequently, the cumulative semantic evolution is strictly bounded by the integral over the finite domain, i.e., $\Lambda(T) = \int_0^T v(t) dt \leq K \cdot T < \infty$.

B.2. Proof of Remark 3.4

Proof We treat the temporal segment as a uniform rod with length l and mass 1, where the centroid is denoted by c . Let $\delta = \text{dist}(x, c)$ represent the deviation of the sampling point from the centroid. According to the parallel axis theorem, the moment of inertia generated by an arbitrary point x (denoted as E_{arb}) is equal to the moment of inertia around the centroid (denoted as E_{opt}) plus the product of the mass and the squared distance from the centroid to that point.

$$E_{\text{arb}} = E_{\text{opt}} + l \cdot \delta^2 \quad (39)$$

Here, based on Equation 32, we have $E_{\text{opt}} = \frac{l^3}{12}$. If the extreme endpoint of the interval is selected, the deviation becomes $\delta = \frac{l}{2}$, yielding

$$E_{\text{worst}} = \frac{l^3}{12} + l \cdot \left(\frac{l}{2}\right)^2 \quad (40)$$

$$E_{\text{worst}} = \frac{l^3}{12} + l \cdot \frac{l^2}{4} = \frac{l^3}{12} + \frac{3l^3}{12} = \frac{4l^3}{12} \quad (41)$$

which implies

$$E_{\text{worst}} = 4 \cdot E_{\text{opt}} \quad (42)$$

If a point is sampled uniformly at random within the interval, the expected value of the squared deviation $E[\delta^2]$ for a uniform distribution is $\frac{l^2}{12}$. Consequently

$$E_{\text{arb}} = E_{\text{opt}} + l \cdot E[\delta^2] = \frac{l^3}{12} + l \cdot \frac{l^2}{12} = \frac{2l^3}{12} \quad (43)$$

which implies

$$E_{\text{arb}} = 2 \cdot E_{\text{opt}} \quad (44)$$

C. Related Domain Knowledge

C.1. Case Information: Three-phase Flow process

The three-phase flow process aims to provide controlled and measurable water, oil, and air flows for a pressurized system, including pipes with different diameters and geometries, as well as a gas-liquid two-phase separator at the top of the high platform(Ruiz-Cárcel et al., 2015). The entire process is mainly divided into five parts: air supply, water and oil supply, mixing and flow, separation, and product recovery. The knowledge base includes the information of 4 common faults (covering *air pipeline blockage*, *water pipeline blockage*, *top separator inlet blockage*, and *2-inch pipeline abnormal pressurization*) and 12 variables (covering *locations*, *physical properties*, *states*, *interaction relationships with other variables*, etc.). We compiled the following fault information for each category to serve as prompts.

1. **Fault Type 1 - Air Pipe Blockage:** This fault simulates an obstruction in the air supply line by manipulating the manual valve located upstream of the mixing point. Under normal conditions, the valve remains fully open to maintain standard airflow. To introduce the fault, the operator gradually closes the valve over time. This process replicates the reduction of available flow channels typically caused by impurity deposition or pipe narrowing. Consequently, the system experiences a progressive decrease in air flow, reflecting the physical characteristics of a developing pneumatic blockage.
2. **Fault Type 2 - Water Pipeline Blockage:** The water pipeline blockage is generated using a similar protocol but applied to the water supply line before the mixing point. By gradually closing the manual water valve, the experiment reproduces flow area reduction caused by scaling, particle accumulation, or emulsified deposits. It is crucial to distinguish this liquid blockage from the air fault because water possesses significantly higher density and viscosity. These physical properties result in more pronounced pressure fluctuations and distinct flow response dynamics during the obstruction process.
3. **Fault Type 3 - Top Separator Inlet Blockage:** This fault occurs at the top separator inlet where the mixed stream enters the system. The simulation involves gradually closing the manual ball valve at the input location. Due to the inherent nonlinear mechanical behavior of ball valves, the pressure drop is negligible when the valve is near the fully open position, but becomes extremely sensitive as it approaches the closed position. This characteristic causes the blockage severity to escalate rapidly in the later stages, resulting in abnormal total flow and pressure distribution entering the separator.
4. **Fault Type 4 - 2 Inch Pipeline Abnormal Pressurization:** The final fault type addresses the accidental pressurization of the two-inch pipeline branch located at the top of the riser. This condition arises when the bridge valve positioned before the top separator opens unexpectedly. Consequently, the originally isolated branch becomes pressurized by forces from the main circuit or the front section of the separator. While this fault does not directly impede the flow of the main four-inch pipe, it significantly alters the local pressure distribution and modifies the operating status of the mixing zone.

The three-phase flow dataset was constructed by Sheng et al., simulating typical faults that may be encountered in real systems (Ruiz-Cárcel et al., 2015). For each fault type, different cases were obtained by simulating different fault lasting times, different fault degrees, and different fault removal times. A total of 11 cases are evaluated under four fault conditions: *air pipeline blockage* (case 1-case 3), *water pipeline blockage* (case 4-case 6), *top separator inlet blockage* (case 7-case 9), and *2-inch pipeline abnormal pressurization* (case 10).

C.2. Case Information: Gas Turbine Power Generation

We collected the time series data of the gas turbine in a real thermal power process. The gas turbine power generation process is designed to convert the chemical energy of natural gas into electrical energy through a thermodynamic cycle involving air compression, fuel combustion, and gas expansion. The system is composed of the unit pre-treatment module, compressor, turbine/combustor, and associated auxiliary systems, all operating synergistically to ensure stable and efficient power generation. The knowledge base includes the information of 44 variables and 8 common faults (covering *Abnormal Cooling Air Pressure in the Gas Turbine Compressor*, *Abnormal Pre-Combustion Natural Gas Temperature in the Gas Turbine Fuel Module*, *High Temperature in the Lower Bearing of the Condensate Pump Motor*, *Abnormal Sealing Water Pressure in the Condensate Pump Mechanical Seal*, *Abnormal Low-Pressure (LP) Steam Flow in the Heat Recovery Steam*

Generator, High Temperature in the Feedwater Pump Bearing, High Stator Temperature in the Circulating Water Pump Motor, Abnormal Acceleration in the Gas Turbine Combustion Chamber).

C.3. Source of Task Knowledge

In industrial contexts, the anticipated K_{task} must encompass two primary components: the root causes identified by operators during variable anomalies and the specific adjustments applied to target locations to mitigate such anomalies. To achieve this, Retrieval Augmented Generation is employed, utilizing time series captions to identify the most probable fault locations and underlying causes. Specifically, such domain knowledge typically resides in unstructured formats, including operation reports, accident case studies, equipment maintenance manuals, and expert system literature (Kister, 2011).

Taking the gas turbine power generation scenario as an example, we compiled a dataset from **10** plants affiliated with a major energy corporation. This collection comprises **1003** accident documents covering **73** distinct equipment types, totaling over **1.35 million** Chinese characters. The corpus contains **204** distinct recording fields, including temporal logs, operator details, unit identification, equipment categories, component types, alarm classes, anomaly locations, fault symptoms, root causes, remedial actions, and preventive measures. This constitutes a comprehensive and complex domain knowledge base.

D. Experimental Supplement

D.1. Experimental Setup

Group Relative Policy Optimization (GRPO)(Shao et al., 2024), which is an RL algorithm for LLM generally used to enhance the exploration performance, is selected as the optimization algorithm, and LoRA(Hu et al., 2022) is selected as the parameter update strategy. In LoRA, a new weight matrix ΔW is inserted parallel to the parameter matrix of the base model W_0 . In the forward reasoning process, both W_0 and ΔW will be multiplied by the same input and then added. For datasets in different tasks and fields, different ΔW can be trained targetedly and added to the base model to realize the calling of description tools for different scenarios. All RL experiments are based on the TRL library(von Werra et al., 2020). Training and reasoning are carried out on the NVIDIA H100s. The detailed hyperparameter settings are shown in Table 6.

Table 6. Hyperparameter Settings in Training. This includes the computing hardware, the hyperparameters for the GRPO algorithm, and the LoRA configurations.

Hyperparameter	Assignment
Computing infrastructure	2*H100-80GB GPU
Batch size per device	2
Gradient accumulation steps	2
Num generations	4
Epochs	10
Optimizer	AdamW
Adam beta1	0.9
Adam beta2	0.99
Weight decay	0.1
Max grad norm	0.1
Learning rate	5e-6
Learning rate scheduler	Cosine
Warmup ratio	0.1
Precision	fp16
LoRA rank	16
LoRA alpha	32
LoRA dropout	0.05
LoRA target modules	q_proj, v_proj, o_proj, gate_proj, up_proj, down_proj (layers 20~47)

D.2. Baseline Model

We evaluate Gemini 3 Pro (Gemini Team, Google DeepMind, 2025), GPT-5.2 (OpenAI, 2025), Deepseek V3.2 (Liu et al., 2025), Qwen3 235B (Team, 2025b), GPT-4o (Achiam et al., 2023), Qwen3-VL (Team, 2025b), Seed1.6-VL (Guo et al.,

2025), GLM4.5V (Hong et al., 2025), Time-MQA (Kong et al., 2025), ChatTS (Xie et al., 2025), ITFormer (Lei et al., 2025) for TSC and Downstream diagnosis tasks. All the baseline methods use the same prompt words.

In the comparison experiment group, for the approaches using LLMs and Multimodal LLMs, we directly used these models to describe the time series. The prompt words, except for removing K_T , maintained the same information as the method in this paper. Moreover, we limit the total textual length of the captions generated by each method to be consistent. This aim is to compare the amount of information within a constrained word budget and analyze redundancy in the captions.

For learning-based captioning methods, given that the existing gold standard dataset contains an insufficient number of time series and captions, totaling only 100, we adopted the parameters from the original model to directly describe the time series. Moreover, We add supervised fine-tuning(SFT) baselines of ITFormer-7B, trained in the three-phase flow industrial scenario, to offer a more direct comparison. We expand the training set through data augmentation, where we add noise to the time series and rewrite the text with synonyms. However, considering the parameter size of ITFormer-7B, this still constitutes a few-shot SFT setting. Due to the small dataset size and to avoid overfitting, we tried 3 different fine-tuning settings.

Table 7. Experimental results of supervised fine-tuning(SFT) baselines of ITFormer-7B. “1” denotes the fine-tuning guidelines in the original paper. The framework of ITFormer-7B consists of a frozen time series encoder, an alignment module, and a frozen LLM. The original authors suggest that transferring this model to other domains only requires fine-tuning the intermediate alignment module and freezing the LLM and the time series encoder. “2” denotes the progressive unfreezing and layer-wise learning rate decay strategies (Howard & Ruder, 2018) to update the alignment module. “3” denotes fine-tuning only a part of the parameters in the alignment module.

Setting	ROUGE-L	FActScore
1	0.102	0.073
2	0.138	0.338
3	0.176	0.502
Ours(Qwen2.5-VL-7B)	0.322	0.924

Despite extensive efforts to prevent overfitting, baselines still suffer from severe overfitting. These results in Table 7 explicitly validate the core advantage of CLFL: our paradigm is highly robust and irreplaceable in realistic industrial scenarios where traditional SFT fails due to high annotation costs.

The purpose of comparing with Zero-shot baselines like GPT-4o and GPT-5.2 is to show that, in time-series description tasks, our proposed RL optimization method enables open-source models with orders of magnitude fewer parameters(only 7B) to match or exceed the performance of advanced large-scale general models. Particularly for scenarios involving data privacy protection, the proposed method serves as a highly cost-effective, locally deployable, and interpretable solution for time-series analysis. The models, such as GPT-4o are closed-source, and their parameters cannot be fine-tuned. We conducted a comparative validation on the same base models. Specifically, we trained Qwen2.5-7B and Qwen2.5-VL-7B. The experimental results in Table 1 directly proves the methodological effectiveness and advantages of the proposed CLFL.

D.3. Human Expert Annotation

To ensure the reliability and accuracy of the dataset, we implemented a rigorous two-stage annotation process. This process involves human experts who generate high-quality descriptions for time series data. We divided the experts into two independent groups to maintain objectivity and robustness. The first group focuses on computational feature extraction, while the second group focuses on semantic interpretation and consensus.

1) Computational Feature Extraction: The primary goal of the first group is to convert raw time series data into structured numerical features. In this step, experts rely on their understanding of time series properties, such as stationarity, periodicity, seasonality, and transient dynamics. For each channel of the multivariate time series, experts use our proposed captioning toolset. They select appropriate tools to compute specific metrics. These metrics serve as the engineering basis for the subsequent description. The output of this stage is a detailed report of numerical features and detected patterns for each time series sample. Guided by domain experience and knowledge, experts obtain numerical results that are highly correlated with the dynamic behaviors of each sample.

2) Semantic Interpretation and Consensus: The second group of experts is responsible for generating the final descriptive text. Their workflow follows a calculation-to-interpretation logic. They do not rely solely on visual observation of the raw data. Instead, they base their interpretation on the structured feature report provided by the first group. This ensures that the descriptions are grounded in mathematical evidence rather than subjective intuition. The annotation procedure in this stage

consists of three steps:

1. **Independent Drafting:** Each expert in the second group independently analyzes the feature report. They write a preliminary description that translates the numerical findings into natural language. This isolation prevents cognitive bias and groupthink during the initial interpretation phase.
2. **Group Discussion:** The experts meet to compare their drafts sentence by sentence. They discuss the validity of the detected dynamic patterns and the accuracy of the language used. If the descriptions from all experts align in terms of semantic meaning and factual judgment, the text is adopted into the final dataset.
3. **Consensus and Finalization:** If there is a disagreement, a strict conflict resolution protocol is triggered. First, experts perform verification by comparing the disputed facts against predefined rules. If the predefined rules do not yield a clear verdict, the experts examine the raw signal visualization together or request additional verification. The final decision is made only when all experts agree on the interpretation of the new evidence.

D.4. Textual Quality Evaluation Metrics

To comprehensively evaluate the quality of the generated captions, we employ four distinct metrics covering lexical overlap, semantic consistency, factual precision, and numeric fidelity.

1) ROUGE-L: ROUGE-L evaluates structural similarity based on the Longest Common Subsequence (LCS). It captures sentence-level structure without requiring consecutive matches. Let c be the candidate (generated) text and r be the reference text, with lengths $|c|$ and $|r|$ respectively. Let $LCS(c, r)$ denote the length of their longest common subsequence. The metric is calculated as the F-measure:

$$\text{ROUGE-L} = \frac{(1 + \beta^2) \cdot LCS(c, r)}{|r| + \beta^2 \cdot |c|} \quad (45)$$

where β is a weighting parameter (typically set to 1) that balances the importance of precision and recall.

2) DeBERTa Score: To assess semantic consistency, we utilize the DeBERTa Score, which applies the BERTScore algorithm using a DeBERTa encoder. This computes the similarity between tokens using contextual embeddings. Given the normalized embedding vectors for the candidate text $\mathbf{c} = \{\mathbf{e}_i\}_{i=1}^{|c|}$ and the reference text $\mathbf{r} = \{\mathbf{e}_j\}_{j=1}^{|r|}$, we compute the Recall (R_{DeB}) and Precision (P_{DeB}) via greedy matching of cosine similarities:

$$R_{\text{DeB}} = \frac{1}{|r|} \sum_{\mathbf{e}_j \in \mathbf{r}} \max_{\mathbf{e}_i \in \mathbf{c}} (\mathbf{e}_i^\top \mathbf{e}_j), \quad P_{\text{DeB}} = \frac{1}{|c|} \sum_{\mathbf{e}_i \in \mathbf{c}} \max_{\mathbf{e}_j \in \mathbf{r}} (\mathbf{e}_i^\top \mathbf{e}_j) \quad (46)$$

The final scalar score is the harmonic mean (F1-measure) of these values:

$$F1_{\text{DeB}} = 2 \cdot \frac{P_{\text{DeB}} \cdot R_{\text{DeB}}}{P_{\text{DeB}} + R_{\text{DeB}}} \quad (47)$$

3) FActScore: FActScore (Factual precision in Atomicity Score) quantifies factual accuracy by decomposing the generated text y into a set of atomic facts $\mathcal{A}_y$. Each atomic fact $a \in \mathcal{A}_y$ is verified against a reliable knowledge source $\mathcal{C}$ (e.g., Wikipedia). The score is defined as the support rate:

$$\text{FActScore}(y) = \frac{1}{|\mathcal{A}_y|} \sum_{a \in \mathcal{A}_y} \mathbb{I}(a \text{ is supported by } \mathcal{C}) \quad (48)$$

where $\mathbb{I}(\cdot)$ is an indicator function equal to 1 if the fact is supported, and 0 otherwise.

4) Numeric Fidelity Score: This metric evaluates the accuracy of numerical values. First, all numerical values are extracted from the candidate set N_c and the reference set N_r . A generated value $v_c \in N_c$ is considered a match to a reference value $v_r \in N_r$ if the relative error is within a threshold (e.g., 5%): $|v_c - v_r|/|v_r| \leq 0.05$. Based on the matching results, we calculate Accuracy (Acc , precision of generated numbers) and Recall (Rec , coverage of reference numbers). The final score is a weighted harmonic mean:

$$\text{Numeric Fidelity} = \frac{1}{\lambda_{Acc} \cdot \frac{1}{Acc} + \lambda_{Rec} \cdot \frac{1}{Rec}} \quad (49)$$

where we set $\lambda_{Rec} = 0.7$ and $\lambda_{Acc} = 0.3$ to penalize the omission of critical data points more heavily than precision errors.

D.5. Component Analysis

In this section, we analyze the necessity of the key components in the CLFL paradigm. Specifically, we ablate the adaptive L_{min} estimation (Appendix A.2 details the calculation of the minimum semantic length), the aggregation under the optimal evolutionary stage partitioning strategy (equidistant partitioning on $\Lambda(T)$), and the dual loop framework.

D.5.1. EFFECT OF ADAPTIVE L_{min} ESTIMATION

We conduct quantitative experiments to analyze the contribution of the adaptive L_{min} estimation relative to a simpler baseline with a fixed window. We have compared the adaptive L_{min} estimation with several fixed-window baselines. We calculate the average length(582, rounded to the nearest integer), the minimum length(83), and the maximum length(2512) in our released TFTSC dataset. Then, based on this, the fixed-length baselines are determined. We then sampled and generated separate datasets for each setting to train Qwen2.5-VL-7B. Experimental results in Table 8 demonstrate that downstream task performance under fixed-length settings is inferior to our proposed adaptive estimation method.

Table 8. Generated caption utility influenced by different L_{min} settings. We sampled and generated separate datasets for each setting to train Qwen2.5-VL-7B. “Ours” denotes that the dataset sampling follows our method (Appendix A.2).

Method	Diagnosis Accuracy
L_{min} fixed at 83	0.8562
L_{min} fixed at 582	0.9132
L_{min} fixed at 2515	0.8641
Ours	0.9743

This discrepancy arises because different samples exhibit varying dynamics and require different time spans. A fixed length cannot adequately adapt to this variability. Therefore, our proposed method adaptively estimates an appropriate L_{min} based on the fluctuation dynamics of each specific time series instance X^{full} . As long as the length of the time windows extracted from X^{full} exceeds L_{min} , informative temporal semantics will consistently be captured and remain worth describing. The experimental results verify that this adaptive estimation of L_{min} is both reasonable and necessary.

D.5.2. EFFECT OF OPTIMAL STAGE PARTITIONING

We conducted a quantitative experimental analysis to demonstrate that partitioning based on semantic manifolds outperforms simpler partitioning rules such as uniform temporal partitioning or more standard latent space distances. We train Qwen2.5-VL-7B using data sampled by different partitioning strategies and evaluate their performance. The number of partitioned stages remains the same across each strategy.

Table 9. Different partitioning rules effects in downstream utility of generated captions. For the “Standard latent-space distance” baseline, we replace the distance metric with Euclidean distance(L2 norm). Specifically, for a given time series X , we use a pre-trained encoder Φ to map each sliding window X_t to a latent vector $z_t \in \mathbb{R}^d$, without treating it as a point on a manifold. All subsequent operations remain identical to our main method.

Method	Uniform temporal partitioning	Standard latent-space distances	Ours
Diagnosis Accuracy	0.9020	0.9266	0.9743

Experimental results in Table 9 demonstrate that our method significantly outperforms alternatives. This is because industrial time series feature abrupt transitions and non-uniform semantic importance. Uniform partitioning fails by ignoring these

shifts. Furthermore, standard Euclidean space is isotropic; it treats all dimensions equally, which easily and incorrectly smooths out critical local dynamics as noise. While raw sensor readings jump abruptly, the underlying physical fault mechanisms evolve continuously. Our method explicitly models this underlying evolutionary continuity in the latent semantic manifold $\mathcal{M}$, not raw observation space.

D.5.3. IMPACT OF THE DUAL LOOP ARCHITECTURE

Capturing local anomalies and generating high-quality text requires strong reasoning capabilities. Therefore, introducing both the inner and outer loops is necessary and indeed yields significant improvements, revealed in the following experimental results.

Inner Loop: For the inner loop, we compare the downstream performance of captions generated by different models, both with and without participation in the reflective agent workflow and the use of the specialized captioning toolset.

Table 10. Generated caption utility influenced by proposed exploration in inner loop. “wo” denotes that LLM describes time series data directly without computational assistance from external toolsets and reflection. A higher value of metric $\uparrow$ indicates better performance.

Method	Diagnosis Accuracy $\uparrow$
Qwen2.5-VL-7B wo	0.3176
Qwen2.5-VL-7B	0.3976
Qwen3-VL wo	0.4706
Qwen3-VL	0.5882
Seed1.6-VL wo	0.5412
Seed1.6-VL	0.7058
GLM4.5V wo	0.6235
GLM4.5V	0.6471
Average Improvement Percentage	18.50%

As shown in Table 10 and Figure 4, the proposed exploration in the inner loop proves to be well designed. Without the reflective agent workflow and pre-defined scope of the time series description toolset, the model tends to describe arbitrarily, leading to uncontrollable outcomes and the generation of irrelevant or uninformative descriptions. Furthermore, the complexity of the LLM interaction is $O(T^2 \times L)$, where T is the number of reasoning rounds and L is the number of tokens per round. To constrain L , we proposed to select key variables(Appendix A.4), bounding the overall interaction costs.

Outer Loop: Experimental results in Table 1 and Table 2 demonstrate that the outer loop boosts ROUGE-L by 114.3% and Diagnosis Accuracy by 145.0%. From the perspective of computational complexity, we use GRPO with $n = 4$ sampled outputs per prompt. This requires only 4 inference passes and removes the need for a large Value Model (unlike PPO). Compared to the hyperparameter settings in other works using GRPO(typically set to 32, 64, or larger), this overhead is acceptable.

Furthermore, calculating contributions based on Integrated Gradients in the Expression Efficiency Reward introduces additional computational cost. To comprehensively analyze its computational cost and robustness, we **ablated Expression Efficiency Reward and replaced the contribution calculation method with SHAP** (SHapley Additive exPlanations), another widely used attribution method based on perturbation. We then retrained the Qwen2.5-VL-7B model and evaluated its performance. Experiments were conducted on a server equipped with two H100 (80GB) GPUs, and Table 11 presents the results.

The standard Shapley computation process has a time complexity proportional to $N \times L_{\text{token}}$ (where N is the number of random permutations of the token sequence). This leads to a computational explosion as text length increases. Empirical tests show that when using Monte Carlo sampling with $N = 10$, a single calculation takes an impractical 8960.65 seconds (see “Ours (Standard SHAP)”). Therefore, it is not feasible to optimize the outer loop of DLAC. To address this, we made a compromise and computed Shapley attribution based on segmented descriptions. Moreover, we have recognized that applying IG to discrete text can introduce out-of-distribution noise during continuous interpolation. To suppress this, our implementation avoids all uninterpretable zero vectors. Instead, we use neutral anchor tokens (e.g.,[PAD] embeddings) as baselines, effectively mitigating the spurious noise. Specifically, we set the number of interpolation steps in “Ours (IG)” to 20. In the “Ours (Segment SHAP)” setup, we used Monte Carlo sampling with $N = 30$. All hyperparameters were determined after convergence testing.

Table 11. Quantitative analysis of the Expression Efficiency Reward. “Without expression-efficiency reward” represents the ablation setting where the expression efficiency reward is removed. “Ours (Segment SHAP)” denotes that attribution computation is based on segmented descriptions. “Ours (IG)” denotes that the expression efficiency reward is computed using the Integrated Gradients attribution method.

Setting	Diagnosis Accuracy	FactScore	Calculation Time (s)
Without expression-efficiency reward	0.8911	0.832	26.0168
Ours (Standard SHAP)	-	-	8960.6506
Ours (Segment SHAP)	0.9659	0.907	73.7494
Ours (IG)	0.9743	0.913	35.3754

As the results show, the substantial gains in Diagnosis Accuracy (89.1% to 97.4%) and text quality outweigh additional but acceptable training-time overhead, which makes the Expression Efficiency Reward necessary. This is because the reward provides a crucial fine-grained supervisory signal. Furthermore, the specific calculation method for the contribution score can indeed be substituted. The perturbation-based methods are computationally prohibitive. Since DLAC optimizes via an outer loop, selecting the IG-based attribution method is significantly more efficient.

D.6. Beyond Classification-oriented Settings

To evaluate whether downstream-task performance is a reliable proxy for caption quality beyond classification-oriented settings, we provide forecasting-support settings. We calculate the statistical correlation between the forecasting error and natural language text quality metrics.

Table 12. Result of statistical correlation. “RO”, “BE”, “FA” indicate the correlation with ROUGE-L metric, BERTScore, and FactScore, respectively. A higher value of metric $\uparrow$ indicates better performance.

Correlation coefficient	RO $\uparrow$	BE $\uparrow$	FA $\uparrow$
ρ	-0.63	-0.61	-0.66

Experimental results in Table 12, based on the Spearman rank correlation coefficient, show that the prediction error MAE has a significant negative correlation with most text quality metrics. Specifically, a smaller error in downstream forecasting tasks corresponds to a higher text quality of the time series descriptions. These results suggest that time series forecasting tasks have the potential to serve as a proxy for evaluating the quality of time series descriptions.

D.7. TSCAgent Generalization

To evaluate the generalization capability of TSCAgent, we employ the model trained on the open-source three-phase flow dataset and assess it within the gas turbine power generation scenario. Specifically, by updating the prompts and knowledge base accordingly, TSCAgent generates descriptions directly for fault diagnosis in the target domain. The test set comprises 1,000 samples with a balanced class distribution. We compare our approach against 3 competitive baselines identified in previous experiments, utilizing the same evaluation metrics as detailed in Table 2.

Table 13. We select a representative and competitive baseline from each category: approaches utilizing LLMs, Multimodal LLMs, and existing supervised methods using caption labels. We compare the performance of our method against these baselines in generating time series captions for the new scenario.

Method	Diagnosis Accuracy
GPT-5.2	0.5580
GPT-4o	0.5237
ITFormer-7B	0.7429
Ours(Qwen2.5-VL-7B)	0.8872

The experimental results are presented in Table 13. These results demonstrate that TSCAgent remains capable of generating captions that are significant for downstream tasks in the gas turbine power generation scenario, even without prior training or fine-tuning on this specific domain. This indicates that the proposed task-driven optimization effectively enhances two

critical capabilities of the agent: the comprehension of time series captioning tools and time series data, and the capacity to understand and apply domain-specific knowledge.

D.8. Empirical Scope Details

We have not only conducted verification on industrial objects. Other settings outside the industrial field are presented in Table 14. The general domain datasets offer greater diversity among samples. This diversity facilitates model training and leads to stronger domain generalization. During the experiments, we observed that the reinforcement learning rewards of the proposed method consistently achieve stable convergence when trained in both specialized and general domains. Furthermore, the method yields satisfactory Classification Accuracy on the respective test sets. These findings demonstrate the robust generality of our proposed method.

Table 14. The empirical objects other than those in industrial settings. The experimental data are derived from open-source data. Here, information such as the dataset source, domain, task introduction, and size of the dataset is presented.

Categories	Dataset Source	Domain	Task Introduction	Experiment Settings
Specialized	PERG-IOBA (Fernández et al., 2024)	Medicine	Retinal disease diagnosis	1200 for training and 154 for test
General	TimeOmni-1-main (Guan et al., 2026)	11 domains including Hydrology, Transportation, Energy, Market, Environment, and Animals	3 types of time-series classification tasks: scenario understanding, decision making, and causality discovery	1300 for training and 243 for test
	TSAIA (Ye et al., 2025)	4 domains including Energy, Weather, Finance, and Medicine.	4 types of time-series classification tasks: climate anomaly detection, ECG anomaly detection, energy anomaly detection, and causality discovery.	

D.9. Scalability

We analyze the scalability of the proposed method from two perspectives: time series length and reflection rounds.

Time series Length: We evaluated the single-sample inference time on a machine equipped with two NVIDIA H100s (80GB). Our proposed reflective agent workflow consists of two main stages. (1) **Describing Phase:** The agent generates tool-calling commands, executes captioning tools, and performs quantitative parsing of the results. (2) **Reflecting Phase:** The agent evaluates the completeness of the generated captions and decides whether a subsequent "Describing" iteration is required. The results across different time series lengths are presented in Figure 7.

Comparing "Ours (Full Describing Phase)" and "Ours (Describing & Reflecting Once)", the inference time increases linearly with the sequence length. This demonstrates that our framework avoids exponential complexity growth, making it acceptable for long-form time series. The runtime of "Ours (Command Gen. Only)" is highly similar to TimeMQA-7B. This is in line with expectations, as both models treat time series data as text input. For LLMs of the same parameter scale (7B), the inference efficiency remains consistent when using similar input modalities. It's worth noting that the time of tool execution and quantitative result parsing is stable. The runtime difference between "Ours(Full Describing Phase)" and "Ours(Command Gen. Only)" stems from tool execution and quantitative result parsing. Since tool computation is fast (millisecond level) and the parsing process is independent of the input sequence length, this overhead remains stable as the time series length increases.

The trade-off between speed and privacy/cost is something worth considering in actual large-scale industrial applications. While closed-source models (GPT-5.2, Deepseek-V3.2) offer faster responses due to massive GPU clusters in companies, they incur high token-based costs. Furthermore, local deployment addresses data privacy concerns and eliminates dependence on external APIs, which is critical for sensitive time series applications.

Reflection iterations: More reflections(length fixed at 600) improve FActScore but increase time. We set different upper limits for the number of reflections and evaluate the average inference time per sample (with a fixed length of 600) as well as the quality of the generated text descriptions for our proposed method ("Ours (Qwen2.5-7B)"). The experimental results in Figure 8 show that an increase in the number of reflection iterations can improve the quality of the text. In practice, the

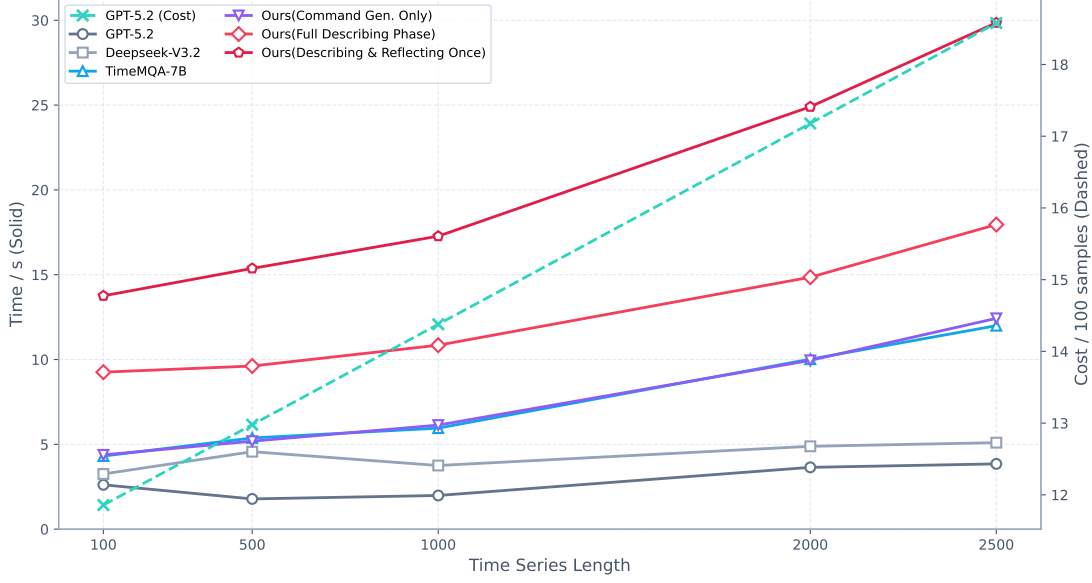


Figure 7. Single-sample inference time (seconds) comparison and economic cost of closed-source LLMs per 100 samples. “GPT-5.2” denotes the GPT-5.2 API service. “Deepseek-V3.2” denotes the Deepseek-V3.2 API service. “TimeMQA-7B” refers to the locally deployed TimeMQA-7B model. “Ours(Command Gen. Only)” represents the time required for the agent to generate tool calling commands. The TSCAgent in the “Ours” experimental group was selected as “Qwen2.5-7B”.

maximum number of reflection iterations can be constrained to flexibly balance performance and speed, as illustrated by point A in the figure.

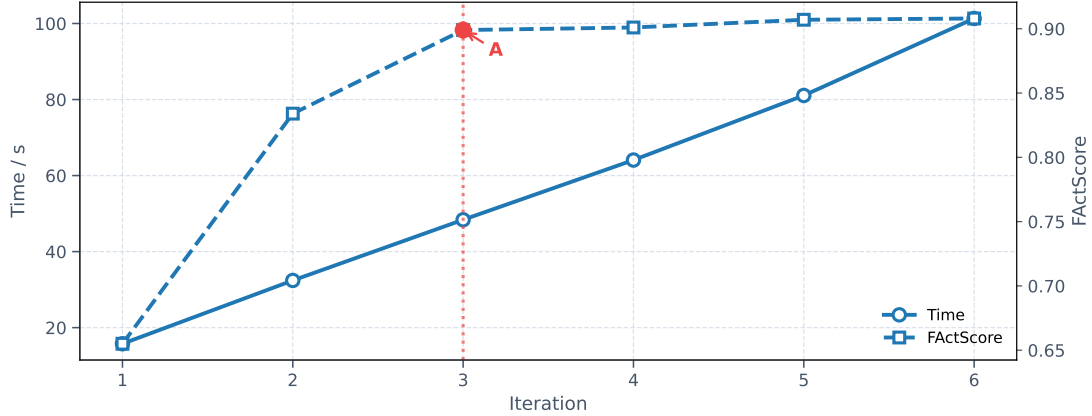


Figure 8. The impact by reflection iteration number on running time and the quality of generated text. The dotted line represents the text quality indicator, and the solid line represents the running time.

D.10. Robustness Evaluation Against Noise

Real-world industrial environments are full of unknown noise. Handling noisy time series data in practical scenarios helps to comprehensively evaluate the effectiveness of our proposed method. We have already validated our method on a noisy, real-world turbine power generation dataset. To provide a more rigorous and quantitative evaluation of our framework’s robustness, we conducted additional controlled noise-injection experiments during the rebuttal period and included these results in our revised manuscript.

We applied additive white Gaussian noise to the original time series from the three-phase flow process at varying Signal-to-Noise Ratio (SNR) levels: 30 dB, 20 dB, and 10 dB. We then used the trained “Ours (Qwen2.5-VL-7B)” model to test its inherent noise resistance on these noisy time series.

Table 15. Robustness performance of the Qwen2.5-VL-7B model under varying Signal-to-Noise Ratio (SNR) levels on the three-phase flow dataset. This table presents a quantitative comparison of the model performance across different noise intensities. The first column categorizes the experimental conditions, ranging from the original noise-free data to heavy noise at 10 dB. The final column provides the percentage reduction in performance for both metrics relative to the original baseline results to illustrate the stability of the framework.

SNR Level	FActScore	Accuracy	Performance Drop
Original(No added noise)	0.913	0.9743	-
30 dB(Mild)	0.892	0.9312	2.4%, 4.4%
20 dB(Moderate)	0.853	0.8788	6.6%, 9.8%
10 dB(Heavy)	0.807	0.8427	11.6%, 13.5%

As the results in Table 15 indicate, the text quality and downstream task utility of the generated captions decreased only marginally (within 15%). This is due to the unique design of the CLFL framework and the perturbations existing in our training data. Since noise is task-irrelevant, the model learns to identify which descriptive perspectives are highly affected by noise and might mislead downstream reasoning, driven by the downstream task reward signal. For example, it prefers stable, macroscopic invariant descriptive perspectives that are robust to local high-frequency noise. Furthermore, it extracts multi-dimensional descriptions to avoid over-reliance on any noise-sensitive perspective. When the SNR is smaller, the time series has been disrupted, and additional denoising methods need to be introduced.

E. Details of TFTSC

E.1. Human Verification

We incorporated a human verification stage to perform post-hoc sanity checks on TFTSC. Instead of relying solely on random sampling, we prioritized samples with low contribution to the downstream task, along with a random subset of high-contribution score samples. Instances containing malformed content or irrelevant text were discarded during validation.

E.2. Example

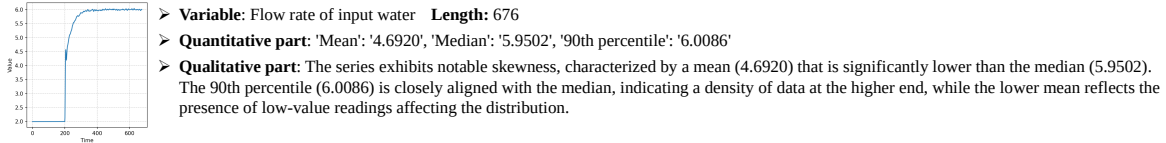


Figure 9. Examples of data in TFTSC illustrating the “Level and Peak” descriptive perspective category. The left shows the time-series data, while the right presents the corresponding captions, consisting of both qualitative and quantitative captions.

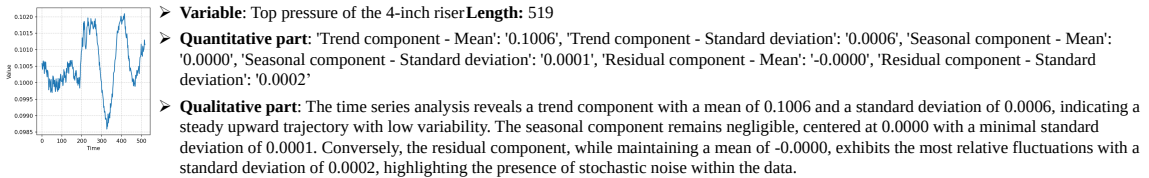


Figure 10. Examples of data in TFTSC illustrating the “STL” descriptive perspective category. The left shows the time-series data, while the right presents the corresponding captions, consisting of both qualitative and quantitative captions.

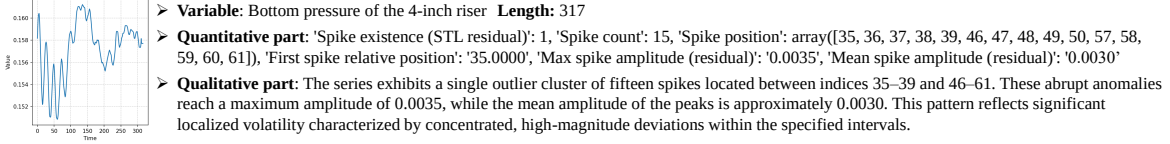


Figure 11. Examples of data in TFTSC illustrating the “Spike” descriptive perspective category. The left shows the time-series data, while the right presents the corresponding captions, consisting of both qualitative and quantitative captions.

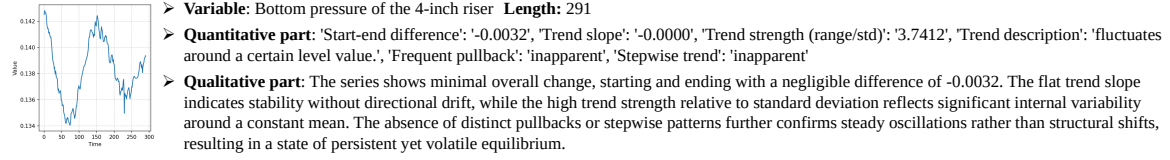


Figure 12. Examples of data in TFTSC illustrating the “Trend” descriptive perspective category. The left shows the time-series data, while the right presents the corresponding captions, consisting of both qualitative and quantitative captions.

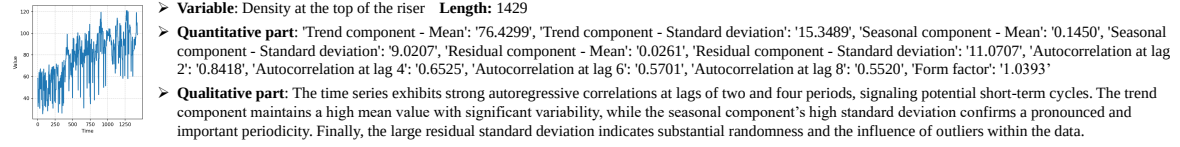


Figure 13. Examples of data in TFTSC illustrating the “Seasonality” descriptive perspective category. The left shows the time-series data, while the right presents the corresponding captions, consisting of both qualitative and quantitative captions.

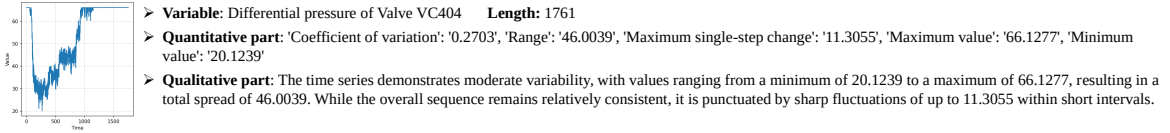


Figure 14. Examples of data in TFTSC illustrating the “Dispersion-Statistical Analysis” descriptive perspective category. The left shows the time-series data, while the right presents the corresponding captions, consisting of both qualitative and quantitative captions.

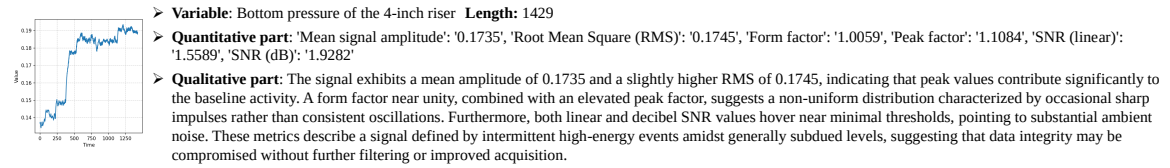


Figure 15. Examples of data in TFTSC illustrating the “Dispersion-Feature Engineer” descriptive perspective category. The left shows the time-series data, while the right presents the corresponding captions, consisting of both qualitative and quantitative captions.

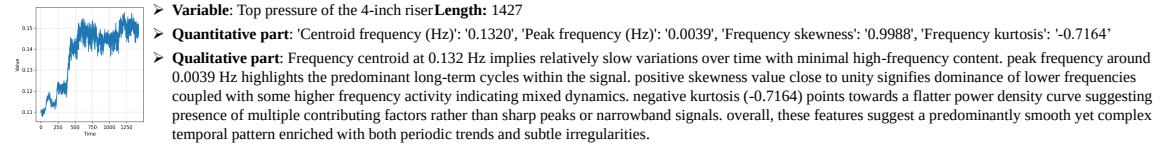


Figure 16. Examples of data in TFTSC illustrating the “Frequency Domain” descriptive perspective category. The left shows the time-series data, while the right presents the corresponding captions, consisting of both qualitative and quantitative captions.

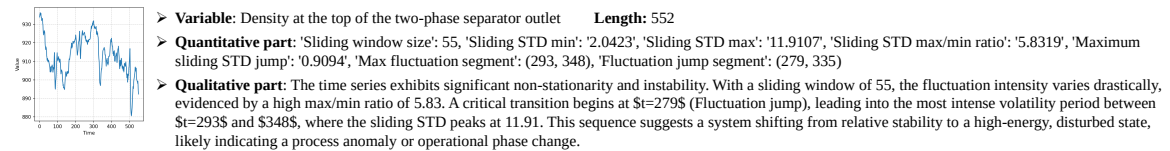


Figure 17. Examples of data in TFTSC illustrating the “Time-Varying Amplitude” descriptive perspective category. The left shows the time-series data, while the right presents the corresponding captions, consisting of both qualitative and quantitative captions.

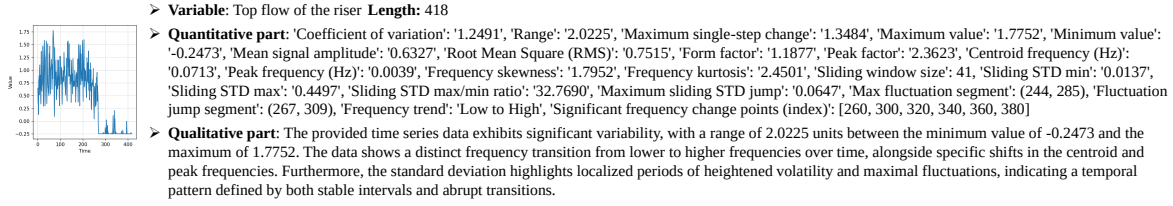


Figure 18. Examples of data in TFTSC illustrating the “Time-Varying Frequency” descriptive perspective category. The left shows the time-series data, while the right presents the corresponding captions, consisting of both qualitative and quantitative captions.

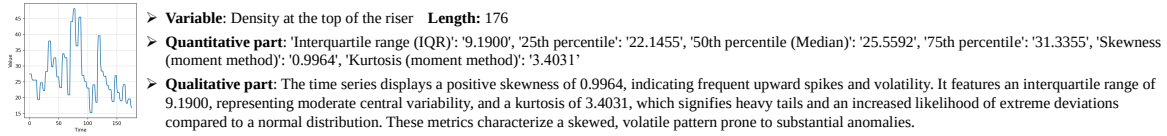


Figure 19. Examples of data in TFTSC illustrating the “Distribution” descriptive perspective category. The left shows the time-series data, while the right presents the corresponding captions, consisting of both qualitative and quantitative captions.

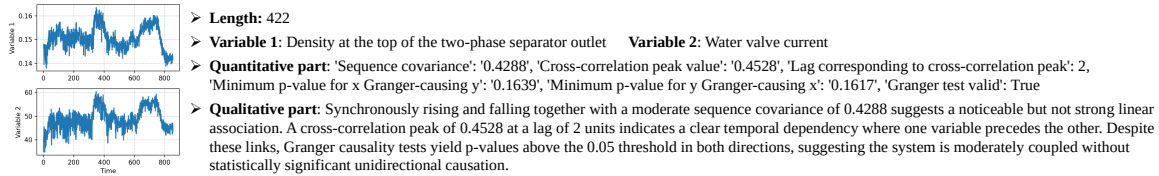


Figure 20. Examples of data in TFTSC illustrating the “Inter-variable” descriptive perspective category. The left shows the time-series data, while the right presents the corresponding captions, consisting of both qualitative and quantitative captions.